UNIDAD 5

OBJETOS Y CLASES AVANZADO

1. RELACIONES ENTRE CLASES.
2. COMPOSICIÓN DE CLASES.
   2.1.Tipos de Composiciones (Composición y Agregación).
3. HERENCIA Y JERARQUÍAS DE CLASES.
   3.1. Cuando Usar Herencia y Cuando Composición.
   3.2. Acceder y Utilizar Miembros Heredados.
   3.3. Modificar y Ampliar Métodos Heredados.
4. ANIDAMIENTO DE CLASES.
5. POLIMORFISMO.
   5.1. Ligadura Estática y Dinámica.
6. CLASES ABSTRACTAS.
7. INTERFACES.
   7.1. Descripción, definición y Uso.
   7.2. Comparación entre Clases Abstractas e Interfaces.
   7.3. Interfaces en Java.
   7.4. Herencia Múltiple con Interfaces.
8. MEJORAS DE INTERFACES EN JAVA 8.
9. EJERCICIOS.

BIBLIOGRAFÍA:

- Java, Cómo Programar (10ª Ed.) Pearson. Paul Deitel y Harvey Deitel (2016).
- Introduction to Programming Using Java (7ª Ed.). David J. Eck (2014)

5.1 RELACIONES ENTRE CLASES.

Cuando vimos el concepto de clase, la describimos como una especie de mecanismo de definición (plantilla o molde), en el que se basa el entorno de ejecución a la hora de construir un objeto: un **mecanismo de definición de objetos**.

Por tanto, a la hora de diseñar un conjunto de clases para **modelar** (representar) el conjunto de información cuyo tratamiento se desea automatizar, es importante establecer apropiadamente las posibles relaciones que puedan existir entre unas clases y otras.

En algunos casos es posible que no exista relación alguna, pero lo más habitual es que sí la haya: una clase puede ser una **especialización** (*relación entre dos clases donde una de ellas (**la subclase**) es una versión más especializada de la otra (**la superclase**), compartiendo características en común pero añadiendo o modificando otras características específicas que la especializan.*

*El punto de vista inverso sería la **generalización** (relación entre dos clases donde una de ellas (la superclase) es una versión más genérica de la otra (la subclase), compartiendo características en común pero sin las propiedades específicas que caracterizan a la subclase (el punto de vista inverso sería la **especialización**).*

Otras relaciones consisten en que **una clase contiene en su interior objetos de otra**, o **una clase utiliza a otra**, etc. Es decir, que entre unas clases y otras habrá que definir cuál es su relación (si es que existe alguna).

Se pueden distinguir diversos tipos de relaciones entre clases:

- **Clientela**. Una clase utiliza objetos de otra clase (por ejemplo

al pasarlos como parámetros a través de sus métodos).

- **Composición**. Alguno de los atributos de una clase es un objeto de otra clase.
- **Anidamiento**. Se definen clases en el interior de otra.
- **Herencia**. una clase comparte determinadas características con otra (clase base), añadiéndole o modificando alguna funcionalidad específica (especialización).

La relación de **clientela** la llevas utilizando desde que has empezado a programar en Java, pues desde tu clase principal (clase con método main) has estado declarando, creando y utilizando objetos de otras clases. Por ejemplo: si utilizas un objeto **String** dentro de la clase principal de tu programa, éste será cliente de la clase **String** (como sucederá con prácticamente cualquier programa que se escriba en Java).

Es la relación fundamental y más habitual entre clases (la utilización de unas clases por parte de otras) y, por supuesto, la que más vas a utilizar tú también, de hecho, ya la has estado utilizando y lo seguirás haciendo.

La relación de **composición** es posible que ya la hayas tenido en cuenta si has definido clases que contenían (tenían como variables miembro) otros objetos en su interior, lo cual es bastante habitual. Por ejemplo, si escribes una clase donde alguno de sus atributos es un objeto de tipo **String**, ya se está produciendo una relación de tipo **composición** (tu clase "tiene" un String, es decir, está compuesta por un objeto String y por algunos elementos más).

La relación de **anidamiento** (o **anidación**) es quizás menos habitual, pues implica declarar unas clases dentro de otras (**clases internas** o

anidadas). En algunos casos puede resultar útil para tener un nivel más de **encapsulamiento** (*ocultamiento del estado de un objeto (de sus datos miembro o atributos)) de manera que sólo se puede cambiar mediante las operaciones (métodos) definidas para ese objeto. Cada objeto está aislado del exterior de manera que se protegen los datos contra su modificación por quien no tenga derecho a acceder a ellos, eliminando efectos secundarios y colaterales no deseados.*

Este modo de proceder permite que el usuario de una clase pueda obviar la implementación de los métodos y propiedades para concentrarse sólo en cómo usarlos. Por otro lado se evita que el usuario pueda cambiar su estado de manera imprevista e incontrolada) y **ocultación** (*efecto que se consigue gracias a la* ***encapsulación****: se evita la visibilidad de determinados miembros de una clase al resto del código del programa para de ese modo comunicarse con los objetos de la clase únicamente a través de su interfaz (métodos)*) de información.

El caso de la relación de **herencia** también es de suma importancia, pues ofrece un mecanismo que aumenta mucho la productividad al ahorrarnos reprogramar de forma innecesaria código.

Podría decirse que tanto la **composición** como la **anidación** son casos particulares de **clientela**, pues en realidad en todos esos casos una clase está haciendo uso de otra (al contener atributos que son objetos de la otra clase, al definir clases dentro de otras clases, al utilizar objetos en el paso de parámetros, declarar variables locales utilizando otras clases, etc.).

5.2 COMPOSICIÓN DE CLASES.

Para indicar que una clase contiene objetos de otra clase no es necesaria ninguna sintaxis especial. Cada uno de esos objetos no es más que un atributo y por tanto debe ser declarado como tal:

```
class nombreClase {
   [modificadores] NombreClase1 nombreAtributo1;
   [modificadores] NombreClase2 nombreAtributo2;
   …
}
```

La clase Punto, define las coordenadas de un punto en el plano 2D:

```
class Punto {
   private x double;
   private y double;

   Punto(double unX, double unY) {
      x = unX;
      y = unY;
   }
   public double getX() { return x; }
   public double getY() { return y; }
}
```

La clase **Rectangulo** define una figura de tipo rectángulo también en el plano 2D a partir de dos de sus vértices (inferior izquierdo y superior derecho). Tal y como hemos formalizado ahora los tipos de relaciones entre clases, parece bastante claro que aquí tendrías un caso de composición: "un rectángulo contiene puntos". Por tanto, podrías definir los atributos de la clase Rectangulo como dos objetos de tipo Punto:

```
class Rectangulo {
   private Punto vertice1;
   private Punto vertice2;
```

```
    …
}
```

EJEMPLO: Intenta implementar los siguientes métodos de la clase Rectangulo teniendo en cuenta su estructura de atributos (dos objetos de la clase Punto):

1 . Método getSuperfice, que calcula y devuelve el área de la superficie encerrada por la figura.

2. Método getPerimetro, que calcula y devuelve la longitud del perímetro de la figura.

```
public double getSuperficie() {
   double area, base, altura; // Variables locales
   base = vertice2.getX() - vertice1.getX();   // longitud lado1
   altura = vertice2.getY() - vertice1.getY(); // longitud lado2
   area = base * altura;
   return (area<0)? -area:area;
}
public double getPerimetro() {
   double perimetro, base, altura; // Variables locales
   base = vertice2.getX() - vertice1.getX();   // longitud lado1
   altura = vertice2.getY() - vertice1.getY(); // longitud lado2
   perimetro = 2 * (base < 0)? -base:base) +
               2 * (altura<0)? -altura:altura);
   return perimetro;
}
```

Preservar la ocultación.

La relación de composición no tiene más misterio a la hora de implementarse que simplemente declarar atributos de las clases que necesites dentro de la clase que estés diseñando.

Ahora bien, cuando escribas clases que contienen objetos de otras clases (lo cual será lo más habitual) deberás tener un poco de precaución con aquellos métodos que devuelvan información acerca de los atributos de la clase (métodos de tipo get).

Lo habitual suele ser declarar los atributos como privados (o protegidos) para ocultarlos a los posibles clientes de la clase (otros objetos que en el futuro harán uso de la clase). Para que otros objetos puedan acceder a la información contenida en los atributos, o al menos a una parte de ella, deberán hacerlo a través de métodos que sirvan de interfaz, de manera que sólo se podrá tener acceso a aquella información que el creador de la clase haya considerado oportuna.

Del mismo modo, los atributos solamente serán modificados desde los métodos de la clase, que decidirán cómo y bajo qué circunstancias deben realizarse esas modificaciones. Con esa metodología de acceso se tenía perfectamente separada la parte de manipulación interna de los atributos de la interfaz con el exterior.

Si los métodos de tipo get devuelven tipos primitivos, devuelven copias del contenido que había almacenado en los atributos, pero los atributos siguen "a salvo" como elementos privados de la clase.

Cuando vayas a devolver un objeto debes ir con mucha precaución. Si en un método de la clase devuelves directamente un objeto que es un atributo, estarás ofreciendo directamente una referencia a un objeto atributo que probablemente has definido como privado. **¡Estás volviendo a hacer público un atributo que inicialmente era privado!**

Para evitar ese tipo de situaciones (ofrecer al exterior referencias a objetos privados) puedes optar por diversas alternativas, procurando siempre evitar la devolución directa de un atributo que sea un objeto:

- Una opción podría ser devolver siempre tipos primitivos.
- Dado que esto no siempre es posible, o como mínimo poco práctico, otra posibilidad es crear un nuevo objeto que sea una

copia del atributo que quieres devolver y utilizar ese objeto como valor de retorno. Es decir, crear una copia del objeto especialmente para devolverlo. De esta manera, el código cliente de ese método podrá manipular a su antojo ese nuevo objeto, pues no será el atributo original, sino un nuevo objeto con el mismo contenido.

- Por último, debes tener en cuenta que es posible que en algunos casos sí se necesite realmente la referencia al atributo original (algo muy habitual en el caso de atributos estáticos). En tales casos, no habrá problema en devolver directamente el atributo para que el código llamante (cliente) lo use como quiera.

- Debes evitar por todos los medios la devolución de un atributo que sea un objeto (estarías dando directamente una referencia al atributo, visible y manipulable desde fuera), salvo que se trate de un caso en el que deba ser así.

Para entender estas situaciones un poco mejor, podemos volver al objeto Rectangulo y observar sus getters.

EJEMPLO: Para la clase Rectangulo, escribe sus métodos getVertice1() y getVertice2() para que devuelvan los vértices inferior izquierdo y superior derecho del rectángulo (objetos de tipo Punto).

```
public Punto getVertice1() { return vertice1; }
public Punto getVertice2() { return vertice2; }
```

Esto funciona perfectamente, pero debes tener cuidado con este tipo de métodos que devuelven directamente una referencia a un objeto atributo que probablemente has definido como privado. Estás de alguna manera, haciendo público el atributo que se declara privado.

Para evitar que esto suceda basta con crear un nuevo objeto que sea una copia del atributo que se desea devolver (en este caso un objeto de la clase Punto). Aquí tienes algunas posibilidades:

```
public Punto getVertice1 () { // Creación de un nuevo punto
   double x, y;
   Punto p;
   x = vertice1.getX();
   y = vertice1.getY();
   p = new Punto(x, y);
   return p;
}

public Punto getVertice2() { // Usar constructor copia de Punto
   Punto p;
   p = new Punto(vertice2);  // (debe definirse) el constructor
   return p;
}
```

De esta manera, se devuelve un punto totalmente nuevo que podrá ser manipulado sin ningún temor por parte del código cliente de la clase pues es una copia para él.

Llamadas a constructores.

Otro factor que debes considerar, a la hora de escribir clases que contengan como atributos objetos de otras clases, es su comportamiento a la hora de instanciarse. Durante el proceso de creación de un objeto (constructor) de la clase contenedora habrá que tener en cuenta también la creación (llamadas a constructores) de aquellos objetos que estén contenidos. El constructor de la clase contenedora debe invocar a los constructores de las clases de los objetos contenidos.

En este caso hay que tener cuidado con las referencias a objetos que se pasan como parámetros para rellenar el contenido de los atributos.

Es conveniente hacer una copia de esos objetos y utilizar esas copias para los atributos pues si se utiliza la referencia que llega del exterior como parámetro, **el código exterior de la clase podría tener acceso a ella sin necesidad de pasar por la interfaz de la clase** (volveríamos a dejar abierta una puerta pública a algo que quizá sea privado).

Además, si el objeto parámetro que se pasó al constructor formaba parte de otro objeto, esto podría ocasionar un desagradable efecto colateral si esos objetos son modificados en el futuro desde el código cliente de la clase, ya que no sabes de dónde provienen esos objetos, si fueron creados especialmente para ser usados por el nuevo objeto o si pertenecen a otro objeto que podría modificarlos más tarde. Es decir, **corres el riesgo de estar "compartiendo" esos objetos con otras partes del código, sin ningún tipo de control de acceso** y con las nefastas consecuencias que eso podría tener: cualquier cambio de ese objeto afectaría a partes del programa supuestamente independientes, que entienden ese objeto como suyo.

En el fondo los objetos no son más que variables de tipo referencia a la zona de memoria en la que se encuentra toda la información del objeto en sí mismo. Esto es, puedes tener un único objeto y múltiples referencias a él. Pero sólo se trata de un objeto, y cualquier modificación desde una de sus referencias afectaría a todas las demás, pues estamos hablando del mismo objeto.

Recuerda también que sólo se crean objetos cuando se llama a un constructor (uso de **new**). Si realizas asignaciones o pasos de parámetros, no se están copiando o pasando copias de los objetos, sino simplemente de las referencias y por tanto se tratará siempre del mismo objeto.

Se trata de un efecto similar al que sucedía en los métodos de tipo get, pero en este caso en sentido contrario (en lugar de que nuestra clase "regale" al exterior uno de sus atributos objeto mediante una referencia, en esta ocasión se "adueña" de un parámetro objeto que probablemente pertenezca a otro objeto y que es posible que en el futuro haga uso de él).

Para entender mejor estos posibles efectos podemos continuar con el ejemplo de la clase Rectangulo que contiene en su interior dos objetos de la clase Punto. En los constructores del rectángulo habrá que incluir todo lo necesario para crear dos instancias de la clase Punto evitando las referencias a parámetros (haciendo copias).

EJEMPLO: Intenta rescribir los constructores de la clase Rectangulo teniendo en cuenta su estructura de atributos (dos objetos de la clase Punto):

1. Un constructor sin parámetros (para sustituir al constructor por defecto) que haga que los valores iniciales de las esquinas del rectángulo sean (0,0) y (1,1).
2. Un constructor con cuatro parámetros (x1, y1, x2, y2) que cree un rectángulo con los vértices (x1, y1) y (x2, y2).
3. Un constructor con dos parámetros (punto1, punto2) que rellene los valores iniciales de los atributos del rectángulo con los valores proporcionados a través de los parámetros.
4. Un constructor con dos parámetros (base, altura), que cree un rectángulo donde el vértice inferior derecho esté ubicado en la posición (0,0) y que tenga una base y una altura tal y como indican los dos parámetros proporcionados.
5. Un constructor de copia (recibe un Rectángulo y crea una copia).

Durante el proceso de creación de un objeto (constructor) de la clase contenedora (en este caso Rectangulo) hay que tener en cuenta también la creación (llamada a constructores) de aquellos objetos que son contenidos (en este caso objetos de la clase Punto). En el caso del primer constructor, habrá que crear dos puntos con las coordenadas (0,0) y (1,1) y asignarlos a los atributos correspondientes (vertice1 y vertice2):

```
public Rectangulo() {
   vertice1 = new Punto(0, 0);
   vertice2 = new Punto(1, 1);
}
```

Para el segundo constructor habrá que crear dos puntos con las coordenadas x1, y1, x2, y2 que han sido pasadas como parámetros:

```
public Rectangulo(double x1, double y1, double x2, double y2) {
   vertice1 = new Punto(x1, y1);
   vertice2 = new Punto(x2, y2);
}
```

En el caso del tercer constructor puedes utilizar directamente los dos puntos que se pasan como parámetros para construir los vértices del rectángulo. Ahora bien, esto podría ocasionar un efecto colateral no deseado si esos objetos de tipo Punto son modificados en el futuro desde el código cliente del constructor (no sabes si esos puntos fueron creados especialmente para ser usados por el rectángulo o si pertenecen a otro objeto que podría modificarlos más tarde).

Por tanto, para este caso quizá sea recomendable crear dos nuevos puntos a imagen y semejanza de los puntos que se han pasado como parámetros. Para ello tendrías dos opciones:

1. Llamar al constructor de la clase Punto con los valores de los atributos (x, y).

2. Llamar al constructor copia de la clase Punto, si es que se dispone de él.

Aquí tienes las posibles versiones:

Se gauardan las referencias que vienen como parámetros, comparte los objetos Punto con el exterior (no son private aunque se declaren):

```
public Rectangulo(Punto vertice1, Punto vertice2) {
   this.vertice1 = vertice1; // this se refiere a este objeto
   this.vertice2 = vertice2; // si no tiene this es el parámetro
}
```

Constructor que "extrae" los atributos de los parámetros y crea nuevos objetos (ahora ya no comparte con el exterior):

```
public Rectangulo(Punto vertice1, Punto vertice2) {
   this.vertice1 = new Punto(vertice1.getX(), vertice1.getY() );
   this.vertice2 = new Punto(vertice2.getX(), vertice2.getY() );
}
```

Constructor que crea los nuevos objetos mediante el constructor copia de la clase Punto (tampoco comparte con el exterior):

```
public Rectangulo(Punto vertice1, Punto vertice2) {
   this.vertice1 = new Punto( vertice1 );
   this.vertice2 = new Punto( vertice2 );
}
```

En este segundo caso puedes observar la utilidad de los constructores de copia a la hora de tener que clonar objetos (algo muy habitual en las inicializaciones).

Para el caso del constructor que recibe como parámetros la base y la altura, habrá que crear vértices con valores (0,0) y (0 + base, 0 + altura), o lo que es lo mismo: (0,0) y (base, altura).

```
public Rectangulo(Punto vertice1, Punto vertice2) {
   this.vertice1 = new Punto(0,0);
```

```
        this.vertice2 = new Punto(base, altura);
    }
```

Quedaría finalmente por implementar el constructor copia:

```
    // Constructor copia de la clase Punto
    public Punto(Punto p) {
        x = p.x;
        y = p.y;
    }
```

EJERCICIO 1: Crea el constructor de copia de la clase Rectangulo.

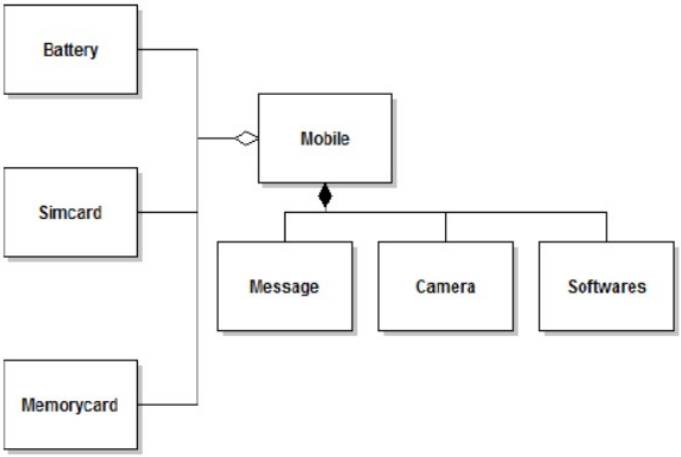
5.2.1. TIPO DE COMPOSICIONES.

En UML se diferencia entre **composición** y **agregación**, dos relaciones entre clases con diferencias muy sutiles.

- **Agregación:** Una clase A está formada por otra clase B (A contiene o agrupa a B). Si el objeto de la clase A deja de existir, el objeto de la clase B puede seguir existiendo. Es una asociación débil.
- **Composición:** una clase A está formada por otra clase B (A contiene o agrupa a B). **Si la clase contenedora desaparece, el objeto contenido también deja de existir.** Es una asociación fuerte. Las partes deben ser privadas.

Si quieres implementar la composición, debes asegurarte que cuando se destruya el objeto, se destruyan también sus partes (eso implica no compartirlas con el exterior).

EJEMPLO: En UML la agregación es un diamante hueco, la composición sombreado:



NOTE:

COMPOSITION

- If the Mobile(Container Object) is destroyed/deleted, then its contents(Contained Object: **Message, Camera, Softwares**) **will be destroyed** beacause of strong-life cycle dependancy

AGGREGATION

- If the Mobile(Container Object) is destroyed/deleted, then its contents(Contained Object: **Battery, Simcard, Memorycard**) **will not be destroyed**

En la composición, un objeto está instanciando a otros objetos dentro de su propio código. Si dices que **Mobile** está compuesto por Message, Camera y Software, eso significa que en algún lado, dentro de la clase Mobile, están escritas las líneas new Message(), new Camera() y new Software() y que no deja accesibles las referencias a estos objetos con el exterior, como mucho les dejará acceder a copias. Por ejemplo:

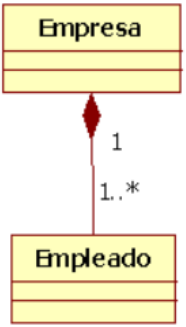
```
public class Mobile {
```

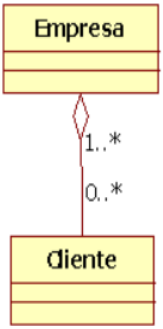
```
        Message message = new Message(); //Compuesto por Message
        Camera camera;
        Software software;

        public Mobile(){ camera = new Camera(); } // Compuesto

        public void initSoftware(){ software = new Software(); }

        class Software { }

        // Si hago esto pongo en peligro la composición
        // Camera se debe crear y destruir desde dentro de la clase
        public Camera getCamera() { return camera; }
    }
```

En caso de hacer cosas como las de amarillo:

```
    public static void Main() {
        Mobile m = new Mobile();
        Message msg = m.message;
        Camera c = m.getCamera();
        m = null; // Prestar atención ahora.
        // Más código... la cámara no se ha destruido, referencia c!!
    }
```

EJERCICIO 2: Observa estos dos gráficos UML e indica dónde se usa la composición y donde la agregación. Intenta implementarlas indicando constructores que reflejen las relaciones.





5.3 HERENCIA Y JERARQUÍAS DE CLASES.

El mecanismo que permite crear clases aprovechando otras que ya existen es conocido como **herencia**. Java implementa la herencia mediante la palabra reservada **extends**.

El concepto de herencia es algo bastante simple y sin embargo muy potente: cuando se desea definir una nueva clase y ya existen clases que, de alguna manera, implementan parte de la funcionalidad que se necesita, es posible crear una nueva clase derivada de la que ya tienes. Al hacer esto, se reutilizan todos los atributos y métodos de la clase previa o clase base (**clase padre** o **superclase**), sin la necesidad de tener que escribirlos de nuevo. Una **subclase** (**derivada** o **hija**) hereda todos los miembros de su clase padre (atributos, métodos y clases internas).

Los constructores no se heredan, aunque se pueden invocar desde la subclase.

Algunos ejemplos de herencia podrían ser:

- Un coche es un vehículo (heredará atributos como la velocidad máxima o métodos como parar y arrancar).
- Un empleado es una persona (heredará atributos como el nombre o la fecha de nacimiento).
- Un rectángulo es una figura geométrica en el plano (heredará métodos como el cálculo de la superficie o de su perímetro).
- Un cocodrilo es un reptil (heredará atributos como por ejemplo el número de dientes).

En este caso la expresión idiomática que puedes usar para plantearte si el tipo de relación entre dos clases A y B es de herencia podría ser

"**es un**": "la clase A es un tipo específico de la clase B" (especialización), o visto de otro modo: "la clase B es un caso general de la clase A" (generalización).

Recuperando algunos ejemplos de clases que ya has utilizado:

- Una ventana en una aplicación gráfica puede ser una clase que hereda de **JFrame** (componente Swing: javax.swing.JFrame), de esta manera esa clase será un marco que dispondrá de todos los métodos y atributos predefinidos de **JFrame** más aquéllos que tú decidas incorporarle al rellenarlo de componentes gráficos.
- Una caja de diálogo puede ser un tipo de **JDialog** (otro componente Swing: javax.swing.JDialog).

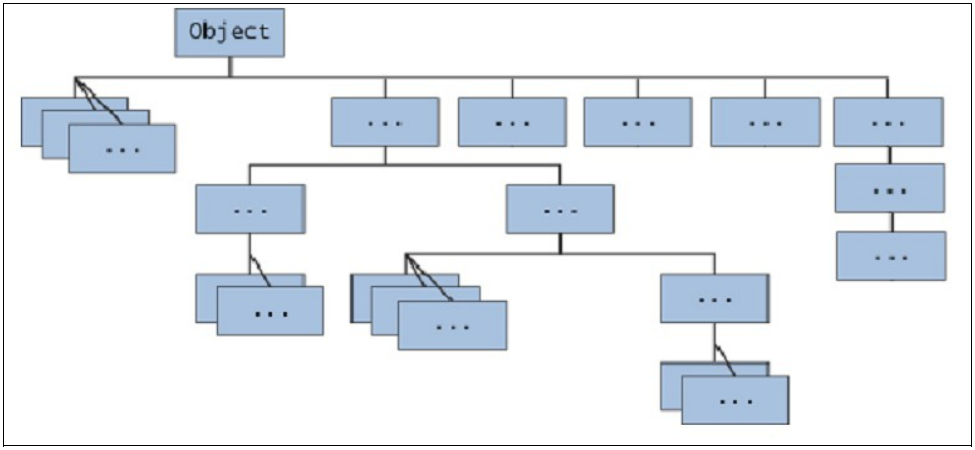


Figura 5: Jerarquía de clases en Java.

En Java, la clase **Object** (dentro del paquete java.lang) define e implementa el comportamiento común a todas las clases (incluidas aquellas que tú escribas). **En Java cualquier clase deriva en última instancia de la clase Object**.

Todas las clases tienen una clase padre, que a su vez también posee

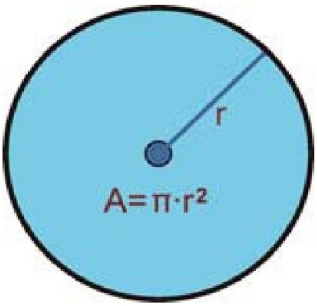
una superclase, y así sucesivamente hasta llegar a la clase **Object**. De esta manera, se construye lo que habitualmente se conoce como una **jerarquía de clases**, que en el caso de Java tendría a la clase Object en la raíz.

5.3.1 CUANDO USAR HERENCIA O COMPOSICIÓN.

Cuando escribas tus propias clases, debes intentar tener claro en qué casos utilizar composición y cuándo herencia:

- **Composición:** cuando una clase está compuesta (tiene partes) por objetos de otras clases. En estos casos se incluyen objetos de esas clases, pero no necesariamente se comparten características con ellos (no se heredan características de esos objetos, sino que directamente se utilizan sus atributos y sus métodos). Esos objetos incluidos no son más que atributos miembros de la clase que se está definiendo.
- **Herencia**: cuando una clase tiene todas las características de otra. En estos casos, la clase derivada es una especialización (o particularización, extensión o restricción) de la clase base. Desde otro punto de vista se diría que la clase base es una generalización de las clases derivadas.

Por ejemplo, imagina que dispones de una clase Punto (ya la has utilizado) y decides definir una nueva clase llamada Círculo. Dado que un punto tiene como atributos sus coordenadas en el plano (x1, y1), decides que es buena idea aprovechar esa información e incorporarla en la clase Circulo que estás escribiendo.



Para ello utilizas la herencia, de manera que al derivar la clase Círculo desde la clase Punto, tendrás disponibles los atributos x1 e y1. Ahora solo faltaría añadirle algunos atributos y métodos más como por ejemplo el radio del círculo, el cálculo de su área y su perímetro, etc.

En principio parece que la idea puede funcionar, pero es posible que más adelante, si continúas construyendo una jerarquía de clases, observes que puedes llegar a conclusiones incongruentes al suponer que un círculo es una especialización de un punto (un tipo de punto). ¿Todas aquellas figuras que contengan uno o varios puntos deberían ser tipos de punto? ¿Y si tienes varios puntos? ¿Cómo accedes a ellos? ¿Un rectángulo también tiene sentido que herede de un punto? No parece muy buena idea.

Parece que en este caso habría resultado mejor establecer una relación de composición. Analízalo detenidamente: ¿cuál de estas dos situaciones te suena mejor?

1. "Un círculo es un punto (su centro)", y por tanto heredará las coordenadas x1 e y1 que tiene todo punto. Además tendrá otras características específicas como el radio o métodos como el cálculo de la longitud de su perímetro o de su área.
2. "Un círculo tiene un punto (su centro)", junto con algunos atributos más como por ejemplo el radio. También tendrá métodos para el cálculo de su área o de la longitud de su perímetro.

Parece que en este caso la composición refleja con mayor sentido la relación que existe entre ambas clases. Normalmente suele ser suficiente con plantearse las preguntas:

"¿A es un tipo de B?" -> Herencia.

"¿A contiene elementos de tipo B?" -> Composición

En Java la herencia se indica mediante la palabra reservada **extends**:

```
[modificadores] class ClasePadre {
  // Cuerpo de la clase ...
}

[modificadores] class ClaseHija extends ClasePadre {
   // Cuerpo de la clase ...
}
```

El día a día de la programación (especialmente para los programadores que trabajan con objetos) consiste en crear subclases: adaptar una clase que ya existe haciendo algunos cambios y añadiendo algo es una situación más común que crear clases desde cero. Para crear una nueva clase a partir de otra se utiliza esta sintaxis:

```
public class clase_nueva extends clase_existente {
  // Elementos cambiados y añadidos
}
```

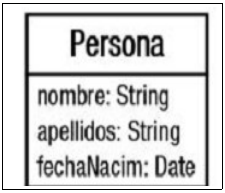
Por ejemplo, imagina que queremos implementar el juego del Blackjack usando las clases **Carta**, **Mano** y **Mazo** que usamos en el tema anterior. Habrá que hacer cambios a la clase **Mano** porque las reglas del juego necesitan poder calcular la puntuación de una mano en cierto momento, que se calcula sumando los valores de las cartas de la mano (el valor de una carta numérica es su valor numérico, y el valor de una sota, caballo y rey es 10 y el valor de un As puede ser tanto 1 como 11. Cuenta como 11 salvo que la suma de todas pase de 21, eso supone que el segundo, tercer y cuarto As de una mano valgan siempre 1).

Una forma de alcanzar este objetivo es extender la clase **Mano** añadiendo un método que calcule su valor en el Blackjack. Ej:

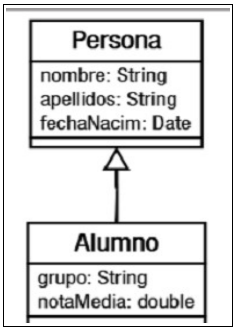
```
public class BlackjackMano extends Mano {
   /**
   * Calcula el valor de la mano en el BlackJack.
   */
   public int getBlackjackValor() {
      int valor;      // El valor calculado
      boolean hayAs; // es true si la mano tiene un as
      int cartas;     // Nº de cartas de la mano
      valor = 0;
      hayAs = false;
      cartas = cuentaCartas();     // método de la clase Mano
      for ( int i = 0; i < cartas; i++ ) {
         // Sumar el valor de la i-ésima carta de la mano
         Carta carta;  // La i-ésima carta
         int v;        // valor de la carta
         carta = dameCarta(i);
         v = carta.getValor(); // El valor normal
         if (v > 10) {
            v = 10;   // para caballo y rey
         }
         if (v == 1) {
            hayAs = true;   // Al menos hay un As
         }
         valor += v;
      }
      // Ahora hay que comprobar si no pasamos de 21 y podemos
      // contar el as como 11
      if ( hayAs && valor + 10 <= 21 )
         valor = valor + 10;
      return val;
   } // fin getBlackjackValor()
} // fin clase BlackjackMano
```

Así que hemos tenido que esforzarnos en implementar un método nuevo, para tener una clase con 8 métodos y variables, reutilizando una clase que ya existía.

Imagina ahora que tienes una clase **Persona** que contiene atributos como nombre, apellidos y fecha de nacimiento:



```
public class Persona {
   String nombre;
   String apellidos;
   GregorianCalendar fechaNacim;
   …
}
```



Es posible que, más adelante, necesites una clase **Alumno** que comparta esos atributos (dado que todo alumno es una persona, pero con algunas características específicas que lo especializan). En tal caso tienes la posibilidad de crear una clase **Alumno** que repita todos esos atributos o bien heredarlos de la clase Persona:

```
public class Alumno extends Persona {
   String grupo;
   double notaMedia;
   …
}
```

A partir de ahora, un objeto de la clase Alumno contendrá los atributos grupo y notaMedia (propios de la clase Alumno), pero también nombre, apellidos y fechaNacim (propios de su clase base Persona y que por tanto ha heredado).

Una clase derivada puede ser a su vez clase padre de otra que herede de ella y así sucesivamente dando lugar a una jerarquía de clases, excepto aquellas que estén en la parte de arriba de la jerarquía (sólo serán clases padre) o en la parte de abajo (sólo serán clases hijas).

Una clase hija no tiene acceso a los miembros privados de su clase padre, tan solo a los públicos (como cualquier parte del código tendría) y los protegidos (a los que sólo tienen acceso las clases derivadas y las del mismo paquete). Aquellos miembros que sean privados en la clase base también habrán sido heredados, pero el acceso a ellos estará

restringido al propio funcionamiento de la superclase y sólo se podrá acceder a ellos si la superclase ha dejado algún medio indirecto para hacerlo (getter o setter).

Todos los miembros de la superclase, tanto atributos como métodos, **salvo los constructores**, son heredados por la subclase. Algunos de estos miembros heredados podrán ser redefinidos o sobrescritos (overriden) y también podrán añadirse nuevos miembros. De alguna manera podría decirse que estás "ampliando" la clase base con características adicionales o modificando algunas de ellas (proceso de especialización). Una clase derivada extiende la funcionalidad de la clase base sin tener que volver a escribir el código de la clase base.

EJEMPLO: Imagina que también necesitas una clase **Profesor**, que contará con atributos como nombre, apellidos, fecha de nacimiento, salario y especialidad. ¿Cómo crearías esa nueva clase y qué atributos le añadirías?

Está claro que un Profesor es otra especialización de Persona, al igual que lo era Alumno, así que podrías crear otra clase derivada de Persona y así aprovechar los atributos genéricos (nombre, apellidos, fecha de nacimiento) que posee todo objeto de tipo Persona. Tan solo faltaría añadirle sus atributos específicos (salario y especialidad):

```
public class Profesor extends Persona {
   String especialidad;
   double salario;
   …
}
```

5.3.2 ACCEDER Y UTILIZAR MIEMBROS HEREDADOS.

Acceso a miembros heredados.

Como ya se ha comentado antes, no es posible acceder a miembros privados de una superclase. Para poder acceder a ellos podrías pensar en hacerlos públicos, pero entonces estarías dando la opción de acceder a ellos a cualquier objeto externo y es probable que tampoco sea eso lo deseable. Para ello se inventó el modificador **protected** (protegido) que permite el acceso desde clases heredadas, pero no desde fuera de las clases (estrictamente hablando, desde fuera del paquete), que serían como miembros privados.

Ya comentamos los posibles modificadores de acceso que podía tener un miembro: sin modificador (acceso de paquete), público, privado o protegido. Aquí tienes de nuevo el resumen:

Cuadro de niveles accesibilidad a los atributos de una clase				
	Misma clase	**Subclase**	**Mismo paquete**	**Otro paquete**
Sin modificador (paquete)	X		X	X
public	X	X	X	X
private	X			
protected	X	X	X	

Figura 6: resumen de modificadores de control de acceso.

Si en el ejemplo anterior de la clase Persona se hubieran definido sus atributos como private:

```
public class Persona {
   private String nombre;
   private String apellidos;
   …
}
```

Al definir la clase Alumno, como heredera de Persona, no habrías tenido acceso a esos atributos, pudiendo ocasionar un grave problema de operatividad al intentar manipular esa información. Por tanto, en estos casos lo más recomendable habría sido declarar esos atributos

como **protected** o bien sin modificador (para que también tengan acceso a ellos otras clases del mismo paquete, si es que se considera oportuno):

```
public class Persona {
   protected String nombre;
   protected String apellidos;
   …
}
```

Sólo en aquellos casos en los que se desea explícitamente que un miembro de una clase no pueda ser accesible desde una clase derivada debería utilizarse el modificador **private**. En el resto de casos es recomendable utilizar **protected**, o bien no indicar modificador (acceso a nivel de paquete).

EJEMPLO: Reescribe las clases **Alumno** y **Profesor** utilizando el modificador **protected** para sus atributos del mismo modo que se ha hecho para su superclase **Persona**.

1. Clase Alumno. Se trata simplemente de añadir el modificador de acceso protected a los nuevos atributos que añade la clase.

```
public class Alumno extends Persona {
   protected String grupo;
   protected double notaMedia;
   …
}
```

2. Clase Profesor. Exactamente igual que en la clase Alumno.

```
public class Profesor extends Persona {
   protected String especialidad;
   protected double salario;
   …
}
```

Utilización de Atributos Heredados.

Los atributos heredados por una clase son, a efectos prácticos, iguales que aquellos que sean definidos específicamente en la nueva clase derivada.

En el ejemplo anterior la clase Persona disponía de tres atributos y la clase Alumno, que heredaba de ella, añadía dos atributos más. Desde un punto de vista funcional podrías considerar que la clase Alumno tiene cinco atributos: tres por ser Persona (nombre, apellidos, fecha de nacimiento) y otros dos más por ser Alumno (grupo y nota media).

EJEMPLO: Dadas las clases **Alumno** y **Profesor** que has utilizado anteriormente, implementa métodos get y set en las clases **Alumno** y **Profesor** para trabajar con sus cinco atributos (tres heredados más dos específicos).

POSIBLE SOLUCIÓN

1. Clase Alumno. Se trata de heredar de la clase Persona y por tanto utilizar con normalidad sus atributos heredados como si pertenecieran a la propia clase (de hecho se puede considerar que le pertenecen, dado que los ha heredado).

```
public class Alumno extends Persona {
   protected String grupo;
   protected double notaMedia;
   // Método getNombre
   public String getNombre(){ return nombre;  }
   // Método getApellidos
   public String getApellidos(){  return apellidos; }
   // Método getFechaNacim
   public GregorianCalendar getFechaNacim(){
      return this.fechaNacim;
   }
   // Método getGrupo
   public String getGrupo(){ return grupo; }
   // Método getNotaMedia
   public double getNotaMedia(){ return notaMedia; }
```

```
        // Método setNombre
        public void setNombre(String nombre){
           this.nombre= nombre;
        }
        // Método setApellidos
        public void setApellidos (String apellidos){
           this.apellidos= apellidos;
        }
        // Método setFechaNacim
        public void setFechaNacim (GregorianCalendar fechaNacim){
           this.fechaNacim= fechaNacim;
        }
        // Método setGrupo
        public void setGrupo (String grupo){
           this.grupo= grupo;
        }
        // Método setNotaMedia
        public void setNotaMedia(double notaMedia){
           this.notaMedia= notaMedia;
        }
    }
```

Si te fijas, puedes utilizar sin problema la referencia **this** al propio objeto con los atributos heredados, pues pertenecen a la clase: **this**.nombre, **this**.apellidos, etc.

2. Clase Profesor. Seguimos exactamente el mismo procedimiento que con la clase Alumno.

```
        public class Profesor extends Profesor {
        String especialidad;
        double salario;
        // Método getNombre
        public String getNombre(){ return nombre; }
        // Método getApellidos
        public String getApellidos(){ return apellidos; }
        // Método getFechaNacim
        public GregorianCalendar getFechaNacim (){
           return this.fechaNacim;
        }
        // Método getEspecialidad
        public String getEspecialidad(){
```

```
            return especialidad;
        }
        // Método getSalario
        public double getSalario(){ return salario; }
        // Método setNombre
        public void setNombre(String nombre){
            this.nombre= nombre;
        }
        // Método setApellidos
        public void setApellidos(String apellidos){
            this.apellidos= apellidos;
        }
        // Método setFechaNacim
        public void setFechaNacim (GregorianCalendar fechaNacim){
            this.fechaNacim= fechaNacim;
        }
        // Método setSalario
        public void setSalario (double salario){
        this.salario= salario;
        }
        // Método setESpecialidad
        public void setESpecialidad (String especialidad){
            this.especialidad= especialidad;
        }
    }
```

Una conclusión que puedes extraer de este código es que has tenido que escribir los métodos get y set para los tres atributos heredados, pero ¿no habría sido posible definir esos seis métodos en la clase base y así estas dos clases derivadas hubieran también heredado esos métodos? La respuesta es afirmativa y de hecho es como lo vas a hacer a partir de ahora.

De esa manera te habrías evitado tener que escribir seis métodos en la clase Alumno y otros seis en la clase Profesor. Así que recuerda: se pueden heredar tanto los atributos como los métodos.

Aquí tienes un ejemplo de cómo podrías haber definido la clase Persona para que luego se hubieran podido heredar de ella sus métodos

(y no sólo sus atributos):

```
public class Persona {
   protected String nombre;
   protected String apellidos;
   protected GregorianCalendar fechaNacim;
   // Método getNombre
   public String getNombre(){ return nombre; }
   // Método getApellidos
   public String getApellidos(){ return apellidos; }
   // Método getFechaNacim
   public GregorianCalendar getFechaNacim(){
      return this.fechaNacim;
   }
   // Método setNombre
   public void setNombre (String nombre){ this.nombre= nombre; }
   // Método setApellidos
   public void setApellidos(String apellidos){
      this.apellidos= apellidos;
   }
   // Método setFechaNacim
   public void setFechaNacim(GregorianCalendar fechaNacim){
      this.fechaNacim= fechaNacim;
   }
}
```

Utilización de Métodos Heredados.

Del mismo modo que se heredan los atributos, también se heredan los métodos, convirtiéndose a partir de ese momento en otros métodos más de la clase derivada, junto a los que hayan sido definidos específicamente.

En el ejemplo de la clase Persona, si dispusiéramos de métodos get y set para cada uno de sus tres atributos (nombre, apellidos, fechaNacim), tendrías seis métodos que podrían ser heredados por sus clases derivadas. Podrías decir entonces que la clase Alumno, derivada de Persona, tiene diez métodos:

- Seis por ser Persona (getNombre, getApellidos, getFechaNacim, setNombre, setApellidos, setFechaNacim).
- Otros cuatro más por ser Alumno (getGrupo, setGrupo, getNotaMedia, setNotaMedia).

Sin embargo, sólo tendrías que definir esos cuatro últimos (los específicos) pues los genéricos ya los has heredado de la superclase.

EJEMPLO: Dadas las clases Persona, Alumno y Profesor que has utilizado anteriormente, implementa métodos get y set en la clase Persona para trabajar con sus tres atributos y en las clases Alumno y Profesor para manipular sus cinco atributos (tres heredados más dos específicos), teniendo en cuenta que los métodos que ya hayas definido para Persona van a ser heredados en Alumno y en Profesor.

POSIBLE SOLUCIÓN

1. Clase Persona.

```
public class Persona {
   protected String nombre;
   protected String apellidos;
   protected GregorianCalendar fechaNacim;
   // Método getNombre
   public String getNombre(){ return nombre; }
   // Método getApellidos
   public String getApellidos(){ return apellidos; }
   // Método getFechaNacim
   public GregorianCalendar getFechaNacim(){
      return this.fechaNacim;
   }
   // Método setNombre
   public void setNombre (String nombre){ this.nombre= nombre; }
   // Método setApellidos
   public void setApellidos(String apellidos){
      this.apellidos= apellidos;
   }
   // Método setFechaNacim
   public void setFechaNacim(GregorianCalendar fechaNacim){
```

```
            this.fechaNacim= fechaNacim;
        }
    }
```

2. Clase Alumno. Al heredar de la clase Persona tan solo es necesario escribir métodos para los nuevos atributos (métodos especializados de acceso a los atributos especializados), pues los métodos genéricos (de acceso a los atributos genéricos) ya forman parte de la clase al haberlos heredado.

```
    public class Alumno extends Persona {
        protected String grupo;
        protected double notaMedia;
        // Método getGrupo
        public String getGrupo(){ return grupo; }
        // Método getNotaMedia
        public double getNotaMedia(){ return notaMedia; }
        // Método setGrupo
        public void setGrupo(String grupo){ this.grupo= grupo; }
        // Método setNotaMedia
        public void setNotaMedia(double notaMedia){
            this.notaMedia= notaMedia;
        }
    }
```

Aquí tienes una demostración práctica de cómo la herencia permite una reutilización eficiente del código, evitando tener que repetir atributos y métodos. Sólo has tenido que escribir cuatro métodos en lugar de diez.

3. Clase Profesor. Seguimos exactamente el mismo procedimiento que con la clase Alumno.

```
    public class Profesor extends Persona {
        String especialidad;
        double salario;
        // Método getEspecialidad
        public String getEspecialidad(){ return especialidad; }
        // Método getSalario
```

```
        public double getSalario(){ return salario; }
        // Método setSalario
        public void setSalario(double salario) {
           this.salario= salario;
        }
        // Método setEspecialidad
        public void setESpecialidad(String especialidad){
           this.especialidad= especialidad;
        }
    }
```

5.3.3 REDEFINIR Y AMPLIAR MÉTODOS.

Redefinir métodos heredados.

Una clase puede redefinir algunos de los métodos que ha heredado de su clase base. En tal caso, el nuevo método (especializado) sustituye al heredado. Este procedimiento también es conocido como de **sobrescritura de métodos**.

En cualquier caso, aunque un método sea sobrescrito o redefinido, aún es posible acceder a él a través de la referencia **super**, aunque sólo se podrá acceder a métodos de la clase padre y no a métodos de clases superiores en la jerarquía de herencia.

Los métodos redefinidos pueden ampliar su accesibilidad con respecto a la que ofrezca el método original de la superclase, pero nunca restringirla. Por ejemplo, si un método es declarado como protected o de paquete en la clase base, podría ser redefinido como public en una clase derivada.

Los métodos estáticos o de clase no pueden ser sobrescritos. Los originales de la clase base permanecen inalterables a través de toda la jerarquía de herencia.

En el ejemplo de la clase Alumno, podrían redefinirse algunos de los

métodos heredados. Por ejemplo, imagina que el método getApellidos devuelva la cadena "Alumno:" junto con los apellidos del alumno. En tal caso habría que reescribir ese método para realizar esa modificación:

```
public String getApellidos () {
   return "Alumno: " + apellidos;
}
```

Cuando sobrescribas un método heredado en Java puedes incluir la anotación **@Override**. En cualquier caso, **no es obligatorio indicar @Override**, pero puede resultar de ayuda a la hora de localizar este tipo de errores (crees que has sobrescrito un método heredado y al confundirte en una letra estás realmente creando un nuevo método diferente). En el caso del ejemplo anterior quedaría:

```
@Override
public String getApellidos() { ...
```

EJEMPLO: Dadas las clases Persona, Alumno y Profesor que has utilizado anteriormente, redefine el método **getNombre** para que devuelva la cadena "Alumno: ", junto con el nombre del alumno, si se trata de un objeto de la clase Alumno o bien "Profesor ", junto con el nombre del profesor, si se trata de un objeto de la clase Profesor.

1. Clase Alumno. Al heredar de la clase Persona tan solo es necesario escribir métodos para los nuevos atributos (métodos especializados de acceso a los atributos especializados), pues los métodos genéricos (de acceso a los atributos genéricos) ya forman parte de la clase al haberlos heredado. Esos son los métodos que se implementaron en el ejercicio anterior (getGrupo, setGrupo, etc.).

Ahora bien, hay que escribir otro método más, pues tienes que redefinir el método getNombre para que tenga un comportamiento un poco diferente al getNombre que se hereda de la clase base Persona:

```
// Método getNombre
@Override
public String getNombre() {
    return "Alumno: " + this.nombre;
}
```

En este caso podría decirse que se "renuncia" al método heredado para redefinirlo con un comportamiento más especializado y acorde con la clase derivada.

2. Clase Profesor. Seguimos exactamente el mismo procedimiento que con la clase Alumno (redefinición del método getNombre).

```
// Método getNombre
@Override
public String getNombre(){
   return "Profesor: " + this.nombre;
}
```

Ampliación de métodos heredados.

Hasta ahora, has visto que para redefinir o sustituir un método de una superclase es suficiente con crear otro método en la subclase que tenga el mismo nombre que el método que se desea sobrescribir. Pero, en otras ocasiones, puede que no necesites sustituir completamente el comportamiento del método de la superclase, sino simplemente ampliarlo.

Para poder hacer esto, necesitas poder preservar el comportamiento antiguo (el de la superclase) y añadir el nuevo (el de la subclase). Para ello, puedes invocar desde el método "ampliador" de la clase derivada al método "ampliado" de la clase superior (teniendo ambos métodos el mismo nombre). ¿Cómo se puede conseguir eso? Puedes hacerlo mediante el uso de la referencia **super**.

La palabra reservada **super** es una referencia a la clase padre de la

clase en la que te encuentres en cada momento (es algo similar a **this**, que representaba una referencia a la clase actual). De esta manera, podrías invocar a cualquier método de tu superclase (si es que se tiene acceso a él).

Por ejemplo, imagina que la clase Persona dispone de un método que permite mostrar el contenido de algunos datos personales de los objetos de este tipo (nombre, apellidos, etc.). Por otro lado, la clase Alumno también necesita un método similar, pero que muestre también su información especializada (grupo, nota media, etc.). ¿Cómo podrías aprovechar el método de la superclase para no tener que volver a escribir su contenido en la subclase?

Podría hacerse de una manera tan sencilla como la siguiente:

```
public void mostrar() {
   super.mostrar(); // Llama al método "mostrar" de la superclase
   // A continuación mUESTRA la información "especializada"
   System.out.printf ("Grupo: %s\n", this.grupo);
   System.out.printf ("Nota media: %5.2f\n", this.notaMedia);
}
```

Este tipo de ampliaciones de métodos resultan especialmente útiles por ejemplo en el caso de los constructores, donde se podría ir llamando a los constructores de cada superclase encadenadamente hasta el constructor de la clase en la cúspide de la jerarquía (el constructor de la clase Object).

EJEMPLO: Dadas las clases Persona, Alumno y Profesor, define un método mostrar para la clase Persona, que muestre el contenido de los atributos (datos personales) de un objeto de la clase Persona. A continuación, define sendos métodos mostrar especializados para las clases Alumno y Profesor que "amplíen" la funcionalidad del método mostrar original de la clase Persona.

1. Método mostrar de la clase Persona.

```
public void mostrar() {
 SimpleDateFormat formatoFecha = new SimpleDateFormat("dd/MM/yyyy");
 String Stringfecha= formatoFecha.format(this.fechaNacim.getTime());
 System.out.printf("Nombre: %s\n", this.nombre);
 System.out.printf("Apellidos: %s\n", this.apellidos);
 System.out.printf("Fecha de nacimiento: %s\n", Stringfecha);
}
```

2. Método mostrar de la clase Profesor. Llamamos al método mostrar de su clase padre (Persona) y luego añadimos la funcionalidad específica para la subclase Profesor:

```
public void mostrar() {
   super.mostrar(); // Llama al método "mostrar" de la superclase
   System.out.printf ("Especialidad: %s\n", this.especialidad);
   System.out.printf ("Salario: %7.2f euros\n", this.salario);
}
```

3. Método mostrar de la clase Alumno. Llamamos al método mostrar de su clase padre (Persona) y luego añadimos la funcionalidad específica para la subclase Alumno:

```
public void mostrar() {
   super.mostrar();
   System.out.printf("Grupo: %s\n", this.grupo);
   System.out.printf("Nota media: %5.2f\n", this.notaMedia);
}
```

Constructores y Herencia.

Recuerda que cuando vimos los constructores se dijo que un constructor de una clase puede llamar a otro constructor de la misma clase, previamente definido, a través de la referencia **this**. En estos casos, la utilización de **this** sólo podía hacerse en la primera línea de código del constructor.

Como ya has visto, un constructor de una clase derivada puede hacer algo parecido para llamar al constructor de su clase base mediante el uso de la palabra **super**. De esta manera, el constructor de una clase derivada puede llamar primero al constructor de su superclase para que inicialice los atributos heredados y posteriormente se inicializarán los atributos específicos de la clase: los no heredados. Nuevamente, esta llamada también debe ser la primera sentencia de un constructor (con la única excepción de que exista una llamada a otro constructor de la clase mediante **this**).

Si no se incluye una llamada a **super()** dentro del constructor, el compilador incluye automáticamente una llamada al constructor por defecto de clase base (llamada a super()). Esto da lugar a una llamada en cadena de constructores de superclase hasta llegar a la clase más alta de la jerarquía (que en Java es la clase Object).

En el caso del constructor por defecto (el que crea el compilador si el programador no ha escrito ninguno), el compilador añade lo primero de todo, antes de la inicialización de los atributos a sus valores por defecto, una llamada al constructor de la clase base mediante la referencia **super**.

A la hora de destruir un objeto (método **finalize**) es importante llamar a los finalizadores en el orden inverso a como fueron llamados los constructores (primero se liberan los recursos de la clase derivada y después los de la clase base mediante la llamada **super.finalize()**).

Si la clase Persona tuviera un constructor de este tipo:

```
public Persona(String nombre, String apellidos, GregorianCalendar
fechaNacim) {
   this.nombe= nombre;
   this.apellidos= apellidos;
```

```
        this.fechaNacim= new GregorianCalendar(fechaNacim);
    }
```

Podrías llamarlo desde un constructor de una clase derivada (por ejemplo Alumno) de la siguiente forma:

```
    public Alumno(String nombre, String apellidos, GregorianCalendar
    fechaNacim, String grupo, double notaMedia) {
        super(nombre, apellidos, fechaNacim);
        this.grupo = grupo;
        this.notaMedia = notaMedia;
    }
```

En realidad se trata de otro recurso más para optimizar la reutilización de código, en este caso el del constructor, que aunque no es heredado, sí puedes invocarlo para no tener que rescribirlo.

EJERCICIO: Escribe un constructor para la clase Profesor que realice una llamada al constructor de su clase base para inicializar sus atributos heredados. Los atributos específicos (no heredados) sí deberán ser inicializados en el propio constructor de la clase Profesor.

```
    public Profesor(String nombre, String apellidos, GregorianCalendar
    fechaNacim, String especialidad, double salario) {
        super(nombre, apellidos, fechaNacim);
        this.especialidad= especialidad;
        this.salario= salario;
    }
```

5.4. ANIDACIÓN DE CLASES.

En algunos lenguajes, es posible definir una clase dentro de otra clase (**clases internas**). En ese caso se dice que la clase está anidada. Las veremos en profundidad en el apartado 5.8. Ejemplo:

```
    class claseContenedora {
        // Cuerpo de la clase…
        class claseInterna {
            // Cuerpo de la clase interna …
```

```
        }
    }
```

Hay varios beneficios al usarlas:

- En primer lugar pones las clases relacionadas juntas en una unidad lógica, ocultándolas al exterior, donde quizás no sean necesarias.
- En segundo lugar, la clase contenedora accede a todos los elementos de sus clases anidadas y viceversa.
- En tercer lugar, simplifican el código.

Por ejemplo se usan mucho con interfaces gráficas como AWT/Swing/.

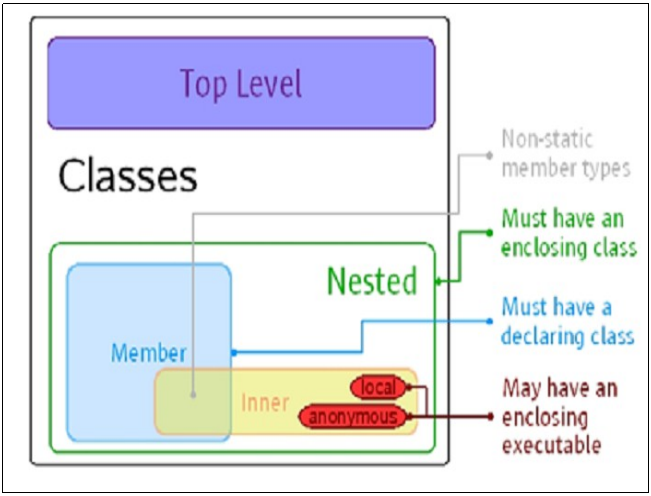


Figura 7: Tipos de clases internas anidadas.

5.5. POLIMORFISMO.

Es otro de los grandes pilares de la Programación Orientada a Objetos (junto con la encapsulación y la herencia). Se trata nuevamente de otra forma más de establecer diferencias entre interfaz e implementación, es decir, entre el qué y el cómo.

La encapsulación te ha permitido agrupar características (atributos) y comportamientos (métodos) dentro de una misma unidad (la clase), pudiendo darles una mayor o menor visibilidad, y permitiendo separar al máximo posible la interfaz de la implementación.

Por otro lado la herencia te ha proporcionado la posibilidad de tratar a los objetos como pertenecientes a una jerarquía de clases. Esta capacidad va a ser fundamental a la hora de poder manipular muchos posibles objetos de clases diferentes como si fueran de la misma clase (**polimorfismo**).

El polimorfismo te va a permitir mejorar la organización y la legibilidad del código así como la posibilidad de desarrollar aplicaciones que sean más fáciles de ampliar a la hora de incorporar nuevas funcionalidades. Si la implementación y la utilización de las clases es lo suficientemente genérica y extensible será más sencillo poder volver a este código para incluir nuevos requerimientos.

Concepto de polimorfismo.

El polimorfismo consiste en la capacidad de poder utilizar una referencia a un objeto de una determinada clase como si fuera de otra clase (en concreto una subclase). Es una manera de decir que una clase podría tener varias (poli) formas (morfismo).

Un **método "polimórfico"** ofrece la posibilidad de ser distinguido (saber a qué clase pertenece) en tiempo de ejecución en lugar de en tiempo de compilación. Para poder hacer algo así es necesario utilizar métodos que pertenecen a una superclase aunque en cada subclase se implementan de una forma particular. En tiempo de compilación se invocará al método sin saber exactamente si será el de una subclase u otra (pues se está invocando al de la superclase). Sólo en tiempo de

ejecución (una vez instanciada una u otra subclase) se conocerá realmente qué método (de qué subclase) es el que finalmente va a ser invocado.

Esta forma de trabajar te va a permitir hasta cierto punto "desentenderte" del tipo de objeto específico (subclase) para centrarte en el tipo de objeto genérico (superclase). De este modo podrás manipular objetos hasta cierto punto "desconocidos" en tiempo de compilación y que sólo durante la ejecución del programa se sabrá exactamente de qué tipo de objeto (subclase) se trata.

El polimorfismo puede llevarse a cabo tanto con superclases (abstractas o no) como con interfaces.

Dada una superclase X, con un método m() y dos subclases A y B, que redefinen ese método m(), podrías declarar un objeto O de tipo X que durante la ejecución podrá ser de tipo A o de tipo B (algo desconocido en tiempo de compilación). Esto significa que al invocarse el método m() de X (superclase), se estará en realidad invocando al método m() de A o de B (alguna de sus subclases). Por ejemplo:

```
// Declaración de una referencia a un objeto de tipo X
ClaseX x; // Objeto de tipo X (superclase)
...
// Zona del programa donde se instancia un objeto de tipo A
(subclase) y se le asigna a la referencia obj.
// La variable obj adquiere la forma de la subclase A.
x = ClaseA();
...
// Otra zona del programa.
// Aquí se instancia un objeto de tipo B (subclase) y se le asigna
// a la referencia obj.
// La variable obj adquiere la forma de la subclase B.
x = ClaseB();
...
// Zona donde se utiliza el método m sin saber realmente qué
```

```
// subclase se está utilizando.
// (Sólo se sabrá durante la ejecución del programa)
x.m() // Llamada al método m (sin saber si será el método m
// de A o de B).
...
```

Imagina que estás trabajando con las clases Alumno y Profesor y que en determinada zona del código podrías tener objetos, tanto de un tipo como de otro, pero eso sólo se sabrá según vaya discurriendo la ejecución del programa. En algunos casos, es posible que un determinado objeto pudiera ser de la clase Alumno y en otros de la clase Profesor, pero en cualquier caso serán objetos de la clase Persona. Eso significa que la llamada a un método de la clase Persona (por ejemplo devolverContenidoString) en realidad será en unos casos a un método (con el mismo nombre) de la clase Alumno y, en otros, a un método (con el mismo nombre también) de la clase Profesor. Esto será posible hacerlo gracias a la ligadura dinámica.

5.5.1 LIGADURA ESTÁTICA Y DINÁMICA.

La conexión que tiene lugar durante una llamada a un método suele ser llamada ligadura (conexión o vinculación que tiene lugar durante una llamada a un método para saber qué código debe ser ejecutado. Puede ser estática o dinámica), vinculación o enlace (en inglés binding).

Si esta vinculación se lleva a cabo durante el proceso de compilación, se le suele llamar **ligadura estática** (la vinculación que se produce en la llamada a un método con la clase a la que pertenece ese método se realiza en tiempo de compilación. Es decir, que antes de generar el código ejecutable se conoce exactamente el método (a qué clase pertenece) que será llamado. También conocido como **vinculación temprana**). En los lenguajes tradicionales, no orientados a objetos, ésta es la única forma de poder resolver la ligadura (en tiempo de compilación).

Sin embargo, en los lenguajes orientados a objetos existe otra posibilidad: la **ligadura dinámica** (la vinculación que se produce en la llamada a un método con la clase a la que pertenece ese método se realiza en tiempo de ejecución). Es decir, que al generar el código ejecutable no se conoce exactamente el método (a qué clase pertenece) que será llamado. Sólo se sabrá cuando el programa esté en ejecución. También conocida como **vinculación tardía**, enlace tardío o **late binding**).

La ligadura dinámica hace posible que sea el tipo de objeto instanciado (obtenido mediante el constructor finalmente utilizado para crear el objeto) y no el tipo de la referencia (el tipo indicado en la declaración de la variable que apuntará al objeto) lo que determine qué versión del método va a ser invocada. El tipo de objeto al que apunta la variable de tipo referencia sólo podrá ser conocido durante la ejecución del programa y por eso el polimorfismo necesita la ligadura dinámica.

En el ejemplo anterior de la clase X y sus subclases A y B, la llamada al método m() sólo puede resolverse mediante ligadura dinámica, pues es imposible saber en tiempo de compilación si el método m() que debe ser invocado será el definido en la subclase A o el definido en la subclase B:

```
// Llamada al método m (sin saber si será el método de A o de B).
x.m() // Resuelta en tiempo de ejecución (ligadura dinámica)
```

EJEMPLO 1: Imagina una clase que representa un instrumento musical genérico (Instrumento) y dos subclases que representen tipos de instrumentos específicos (por ejemplo Flauta y Piano). Todas las clases tendrán un método tocarNota(), que será específico para cada subclase. Haz un pequeño programa de ejemplo en Java que utilice el polimorfismo (referencias a la superclase que se convierten en

instancias específicas de subclases) y la ligadura dinámica (llamadas a un método que aún no están resueltas en tiempo de compilación) con estas clases que representan instrumentos musicales. Puedes implementar el método tocarNota() mediante la escritura de un mensaje en pantalla.

La clase Instrumento podría tener un único método (tocarNota):

```
public class Instrumento {
    public void tocarNota (String nota) {
       System.out.printf("Instrumento: tocar nota %s.\n", nota);
    }
}
```

En el caso de las clases Piano y Flauta puede ser similar, heredando de Instrumento y redefiniendo el método tocarNota:

```
public class Flauta extends Instrumento {
   @Override
   public void tocarNota(String nota) {
      System.out.printf ("Flauta: tocar nota %s.\n", nota);
   }
}
public class Piano extends Instrumento {
   @Override
   public void tocarNota (String nota) {
      System.out.printf ("Piano: tocar nota %s.\n", nota);
   }
}
```

A la hora de declarar una referencia a un objeto de tipo instrumento, utilizamos la superclase (Instrumento):

```
Instrumento i1; // Ejemplo de objeto polimórfico
```

Sin embargo, a la hora de instanciar el objeto, utilizamos el constructor de alguna de sus subclases (Piano, Flauta, etc.):

```
if (<condición>) {
```

```
        // En este caso va adquirir forma de Piano
        i1= new Piano();
    }
    else if (<condición>2) {
        // En este caso va adquirir forma de Flauta
        i1= new Flauta();
    } else {
        ...
    }
```

Finalmente, a la hora de invocar el método tocarNota(), no se sabe a qué versión (de qué subclase) de tocarNota() se está llamando, pues depende del tipo de objeto (subclase) que se haya instanciado. Se estará utilizando por tanto la ligadura dinámica:

```
// Interpretamos una nota con el objeto i1
// No sabemos si se ejecutará el método de Piano o de Flauta
// (dependerá de la ejecución)
i1.tocarNota("do"); // Ejemplo de ligadura dinámica
```

Limitaciones de la ligadura dinámica.

Como has podido comprobar, el polimorfismo se basa en la utilización de referencias de un tipo más "amplio" (superclases) que los objetos a los que luego realmente van a apuntar (subclases). Ahora bien, existe una importante restricción en el uso de esta capacidad, pues el tipo de referencia limita cuáles son los métodos que se pueden utilizar y los atributos a los que se pueden acceder.

No se puede acceder a los miembros específicos de una subclase a través de una referencia a una superclase. Sólo se pueden utilizar los miembros declarados en la superclase, aunque la definición que finalmente se utilice en su ejecución sea la de la subclase.

Veamos un ejemplo: si dispones de una clase A que es subclase de B y declaras una variable como referencia un objeto de tipo B. Aunque más

tarde esa variable haga referencia a un objeto de tipo A (subclase), los miembros a los que podrás acceder sin que el compilador produzca un error serán los miembros de A que hayan sido heredados de B (superclase). De este modo, se garantiza que los métodos que se intenten llamar van a existir cualquiera que sea la subclase de B a la que se apunte desde esa referencia.

En el ejemplo de las clases Persona, Profesor y Alumno, el polimorfismo nos permitiría declarar variables de tipo Persona y más tarde hacer con ellas referencia a objetos de tipo Profesor o Alumno, pero no deberíamos intentar acceder con esa variable a métodos que sean específicos de la clase Profesor o de la clase Alumno, tan solo a métodos que sabemos que van a existir seguro en ambos tipos de objetos (métodos de la superclase Persona).

EJEMPLO: Haz un pequeño programa en Java en el que se declare una variable de tipo Persona, se pidan algunos datos sobre esa persona (nombre, apellidos y si es alumno o si es profesor), y se muestren nuevamente esos datos en pantalla, teniendo en cuenta que esa variable no puede ser instanciada como un objeto de tipo Persona (es una clase abstracta) y que tendrás que instanciarla como Alumno o como Profesor. Recuerda que para poder recuperar sus datos necesitarás hacer uso de la ligadura dinámica y que tan solo deberías acceder a métodos que sean de la superclase.

Si tuviéramos diferentes variables referencia a objetos de las clases Alumno y Profesortendrías algo así:

```
Alumno obj1;
Profesor obj2;
...
// Si se dan ciertas condiciones el objeto será de tipo Alumno y
// lo tendrás en obj1
```

```
System.out.printf ("Nombre: %s\n", obj1.getNombre());
// Si se dan otras condiciones el objeto será de tipo Profesor y
// lo tendrás en obj2
System.out.printf ("Nombre: %s\n", obj2.getNombre());
```

Pero si pudieras tratar de una manera más genérica la situación, podrías intentar algo así:

```
Persona obj;
// Si se dan ciertas condiciones el objeto será de tipo Alumno y
// por tanto lo instanciarás como tal
obj = new Alumno(<parámetros>);
// Si se otras condiciones el objeto será de tipo Profesor y por
// tanto lo instanciarás como tal
obj = new Profesor(<parámetros>);
```

De esta manera la variable obj podría contener una referencia a un objeto de la superclase Persona de subclase Alumno o bien de subclase Profesor (polimorfismo). Esto significa que independientemente del tipo de subclase que sea (Alumno o Profesor), podrás invocar a métodos de la superclase Persona y durante la ejecución se resolverán como métodos de alguna de sus subclases:

```
//En tiempo de compilación no se sabrá de qué subclase de Persona
// será obj. Habrá que esperar la ejecución para que el entorno lo
// sepa e invoque al método adecuado.
System.out.printf("Contenido del objeto usuario: %s\n",
stringContenidoUsuario);
```

Por último recuerda que debes de proporcionar constructores a las subclases Alumno y Profesor que sean "compatibles" con algunos de los constructores de la superclase Persona, pues al llamar a un constructor de una subclase, su formato debe coincidir con el de algún constructor de la superclase (como debe suceder en general con cualquier método que sea invocado utilizando la ligadura dinámica).

Interfaces y polimorfismo.

Es posible también llevar a cabo el polimorfismo mediante el uso de interfaces. Un objeto puede tener una referencia cuyo tipo sea una interfaz, pero para que el compilador te lo permita, la clase cuyo constructor se utilice para crear el objeto deberá implementar esa interfaz (bien por si misma o bien porque la implemente alguna superclase).

Un objeto cuya referencia sea de tipo interfaz sólo puede utilizar aquellos métodos definidos en la interfaz, es decir, que no podrán utilizarse los atributos y métodos específicos de su clase, tan solo los de la interfaz.

Las referencias de tipo interfaz permiten unificar de una manera bastante estricta la forma de utilizarse de objetos que pertenezcan a clases muy diferentes (pero que todas ellas implementan la misma interfaz). De este modo podrías hacer referencia a diferentes objetos que no tienen ninguna relación jerárquica entre sí utilizando la misma variable (referencia a la interfaz). Lo único que los distintos objetos tendrían en común es que implementan la misma interfaz. En este caso sólo podrás llamar a los métodos de la interfaz y no a los específicos de las clases.

Por ejemplo, si tenías una variable de tipo referencia a la interfaz Arrancable, podrías instanciar objetos de tipo Coche o Motosierra y asignarlos a esa referencia (teniendo en cuenta que ambas clases no tienen una relación de herencia). Sin embargo, tan solo podrás usar en ambos casos los métodos y los atributos de la interfaz Arrancable (por ejemplo arrancar) y no los de Coche o los de Motosierra (sólo los genéricos, nunca los específicos).

En el caso de las clases Persona, Alumno y Profesor, podrías declarar,

por ejemplo, variables del tipo Imprimible:

```
Imprimible obj; // Imprimible es una interfaz y no una clase
```

Con este tipo de referencia podrías luego apuntar a objetos tanto de tipo Profesor como de tipo Alumno, pues ambos implementan la interfaz Imprimible:

```
// En algunas circunstancias podría suceder esto:
obj= new Alumno(nombre, apellidos, fecha, grupo, nota);
// Polimorfismo con interfaces
...
// En otras circunstancias podría suceder esto:
obj= new Profesor(nombre, apellidos, fecha, especialidad,
salario);
// Polimorfismo con interfaces ... Y más adelante hacer uso de la
ligadura dinámica:
// Llamadas sólo a métodos de la interfaz
String contenido;
contenido= obj.devolverContenidoString(); // Ligadura dinámica
```

Conversión de objetos.

En principio no se puede acceder a los miembros específicos de una subclase a través de una referencia a una superclase. Si deseas tener acceso a todos los métodos y atributos específicos del objeto subclase tendrás que realizar una **conversión explícita (casting)** que convierta la referencia más general (superclase) en la del tipo específico del objeto (subclase).

Para que puedas realizar conversiones entre distintas clases es obligatorio que exista una relación de herencia entre ellas (una debe ser clase derivada de la otra). **Se realizará una conversión implícita o automática de subclase a superclase siempre que sea necesario**, pues un objeto de tipo subclase siempre contendrá toda la información necesaria para ser considerado un objeto de la superclase.

Ahora bien, la conversión en sentido contrario (de superclase a subclase) debe hacerse de forma explícita y según el caso podría dar lugar a errores por falta de información (atributos) o de métodos. En tales casos se produce una excepción de tipo **ClassCastException**. Por ejemplo, imagina que tienes una clase A y una clase B, subclase de A:

```
class ClaseA {
   public int atrib1;
}
class ClaseB extends ClaseA {
    public int atrib2;
}
```

A continuación declaras una variable referencia a la clase A (superclase) pero sin embargo le asignas una referencia a un objeto de la clase B (subclase) haciendo uso del polimorfismo:

```
A obj; // Referencia a objetos de la clase A
obj= new B(); // Referencia a objetos clase A, pero apunta
              // realmente a objeto clase B (polimorfismo)
```

El objeto que acabas de crear como instancia de la clase B (subclase de A) contiene más información que la que la referencia obj te permite en principio acceder sin que el compilador genere un error (pues es de clase A). En concreto los objetos de la clase B disponen de atrib1 y atrib2, mientras que los objetos de la clase A sólo de atrib1. Para acceder a esa información adicional de la clase especializada (atrib2) tendrás que realizar una conversión explícita (casting):

```
// Casting del tipo A al tipo B (funcionará bien porque el objeto
// es realmente del tipo B)
System.out.printf("obj.atrib2=%d\n", ((B) obj).atrib2);
```

Sin embargo si se hubiera tratado de una instancia de la clase A y hubieras intentado acceder al miembro atrib2, se habría producido una excepción de tipo ClassCastException:

```
A obj; // Referencia a objetos de la clase A
obj= new A(); // Referencia a objetos de la clase A, y apunta
realmente a un objeto de la clase A
// Casting del tipo A al tipo B (puede dar problemas porque el
// objeto es realmente del tipo A):
// Funciona (la clase A tiene atrib1)
System.out.printf("obj.atrib2=%d\n", ((B) obj).atrib1);
// ¡Error en ejecución! (la clase A no tiene atrib2). Producirá
// una ClassCastException.
System.out.printf ("obj.atrib2=%d\n", ((B) obj).atrib2);
```

5.6 CLASES ABSTRACTAS.

En determinadas ocasiones, es posible que necesites definir una clase que represente un concepto lo suficientemente abstracto como para que nunca vayan a existir instancias de ella (objetos). ¿Tendría eso sentido? ¿Qué utilidad podría tener?

Imagina una aplicación para un centro educativo que utilice las clases de ejemplo Alumno y Profesor, ambas subclases de Persona. Es más que probable que esa aplicación nunca llegue a necesitar objetos de la clase Persona, pues serían demasiado genéricos como para poder ser utilizados (no contendrían suficiente información específica). Podrías llegar entonces a la conclusión de que la clase Persona ha resultado de utilidad como clase base para construir otras clases que hereden de ella, pero no como una clase instanciable de la cual vayan a existir objetos. A este tipo de clases se les llama clases abstractas.

En algunos casos puede resultar útil disponer de clases que nunca serán instanciadas, sino que proporcionan un marco o modelo a seguir por sus clases derivadas dentro de una jerarquía de herencia. Son las clases abstractas.

La posibilidad de declarar clases abstractas es una de las

características más útiles de los lenguajes orientados a objetos, pues permiten dar unas líneas generales de cómo es una clase sin tener que implementar todos sus métodos o implementando solamente algunos de ellos. Esto resulta especialmente útil cuando las distintas clases derivadas deban proporcionar los mismos métodos indicados en la clase base abstracta, pero su implementación sea específica para cada subclase.

Imagina que estás trabajando en un entorno de manipulación de objetos gráficos y necesitas trabajar con líneas, círculos, rectángulos, etc. Estos objetos tendrán en común algunos atributos que representen su estado (ubicación, color del contorno, color de relleno, etc.) y algunos métodos que modelen su comportamiento (dibujar, rellenar con un color, escalar, desplazar, rotar, etc.). Algunos de ellos serán comunes para todos ellos (por ejemplo la ubicación o el desplazamiento) y sin embargo otros (como por ejemplo dibujar) necesitarán una implementación específica dependiendo del tipo de objeto.

Pero, en cualquier caso, todos ellos necesitan esos métodos (tanto un círculo como un rectángulo necesitan el método dibujar(), aunque se lleven a cabo de manera diferente). En este caso resultaría muy útil disponer de una clase abstracta objeto gráfico donde se definirían las líneas generales (algunos atributos concretos comunes, algunos métodos concretos comunes implementados y algunos métodos genéricos comunes sin implementar) de un objeto gráfico y más adelante, según se vayan definiendo clases especializadas (líneas, círculos, rectángulos), se irán concretando en cada subclase aquellos métodos que se dejaron sin implementar en la clase abstracta.

Declaración de una clase abstracta.

Una clase abstracta es una clase que no se puede instanciar, es decir, que no se pueden crear objetos a partir de ella. La idea es permitir que otras clases deriven de ella, proporcionando un modelo genérico y algunos métodos de utilidad general. Las clases abstractas se declaran mediante el modificador **abstract**:

```
[modificador_acceso] abstract class nombreClase [herencia]
[interfaces] {
   …
}
```

Una clase puede contener en su interior métodos declarados como **abstract** (métodos para los cuales sólo se indica la cabecera, pero no se proporciona su implementación). En tal caso, la clase tendrá que ser necesariamente también **abstract**. **Esos métodos tendrán que ser posteriormente implementados en sus clases derivadas.**

Por otro lado, una clase también puede contener métodos totalmente implementados (no abstractos), los cuales serán heredados por sus clases derivadas y podrán ser utilizados sin necesidad de definirlos (pues ya están implementados).

Cuando trabajes con clases abstractas debes tener en cuenta:

- Una clase abstracta sólo puede usarse para crear nuevas clases derivadas. No se puede hacer un **new** de una clase abstracta. Se produciría un error de compilación.
- Una clase abstracta puede contener métodos totalmente definidos (no abstractos) y métodos sin definir (métodos abstractos).

EJEMPLO: Basándote en la jerarquía de clases de ejemplo (Persona, Alumno, Profesor), que ya se han utilizado en otras ocasiones, modifica

lo que consideres oportuno para que Persona sea, a partir de ahora, una clase abstracta (no instanciable) y las otras dos clases sigan siendo clases derivadas de ella, pero sí instanciables.

En este caso lo único que habría que hacer es añadir el modificador **abstract** a la clase Persona. El resto de la clase permanecería igual y las clases Alumno y Profesor no tendrían porqué sufrir ninguna modificación.

```
public abstract class Persona {
   protected String nombre;
   protected String apellidos;
   protected GregorianCalendar fechaNacim;
   ...
}
```

A partir de ahora no podrán existir objetos de la clase Persona. El compilador generaría un error.

Existen una gran cantidad de clases abstractas en la API de Java. Aquí tienes un par de ejemplos:

- La clase java.awt.Component:

```
public abstract class Component extends Object implements
ImageObserver, MenuContainer, Serializable...
```

- La clase javax.swing. AbstractButton:

```
public abstract class AbstractButton extends JComponent
implements ItemSelectable, SwingConstants...
```

Métodos abstractos.

Un método abstracto es un método declarado en una clase para el cual esa clase no proporciona la implementación. Si una clase dispone de al menos un método abstracto se dice que es una clase abstracta. Toda

clase que herede (sea subclase) de una clase abstracta debe implementar todos los métodos abstractos de su superclase o bien volverlos a declarar como abstractos (y por tanto también sería abstracta). Para declarar un método abstracto en Java se utiliza el modificador **abstract**) es un método cuya implementación no se define, sino que se declara únicamente su interfaz (cabecera) para que su cuerpo sea implementado más adelante en una clase derivada.

Un método se declara como abstracto mediante el uso del modificador abstract (como en las clases abstractas):

[modificador_acceso] **abstract** <tipo> <nombreMetodo> ([parámetros]) [excepciones];

Estos métodos tendrán que ser obligatoriamente redefinidos (en realidad "definidos", pues aún no tienen contenido) en las clases derivadas. Si en una clase derivada se deja algún método abstracto sin implementar, esa clase derivada será también una clase abstracta. **Cuando una clase contiene un método abstracto tiene que declararse como abstracta obligatoriamente.**

Imagina que tienes una clase Empleado genérica para diversos tipos de empleado y 3 clases derivadas: EmpleadoFijo (tiene un salario fijo más ciertos complementos), EmpleadoTemporal (salario fijo más otros complementos diferentes) y EmpleadoComercial (una parte de salario fijo y unas comisiones por cada operación). La clase Empleado podría contener un método abstracto calcularNomina(), pues sabes que el método será necesario para cualquier tipo de empleado (todo empleado cobra una nómina). Sin embargo el cálculo en sí de la nómina será diferente si se trata de un empleado fijo, un empleado temporal o un empleado comercial, y será dentro de las clases especializadas de Empleado (EmpleadoFijo¸ EmpleadoTemporal, EmpleadoComercial)

donde se implementen de manera específica el cálculo de las mismas.

Debes tener en cuenta al trabajar con métodos abstractos:

- Un método abstracto implica que la clase a la que pertenece tiene que ser abstracta, pero eso no significa que todos los métodos de esa clase tengan que ser abstractos.
- Un método abstracto no puede ser privado (no se podría implementar, dado que las clases derivadas no tendrían acceso a él).
- Los métodos abstractos no pueden ser estáticos, pues los métodos estáticos no pueden ser redefinidos (y los métodos abstractos necesitan ser redefinidos).

Puedes echar un vistazo a este vídeo en el se explica el funcionamiento de las clases abstractas y se muestra un ejemplo de creación y utilización: https://www.youtube.com/watch?v=ztpYmmecfQs

EJEMPLO: Basándote en la jerarquía de clases Persona, Alumno, Profesor, crea un método abstracto llamado **mostrar()** para la clase Persona. Dependiendo del tipo de persona (alumno o profesor) el método tendrá que mostrar unos u otros datos personales (habrá que hacer implementaciones específicas en cada clase derivada). Una vez hecho esto, implementa completamente las tres clases (con todos sus atributos y métodos) y utilízalas en un pequeño programa de ejemplo que cree un objeto de tipo Alumno y otro de tipo Profesor, los rellene con información y muestre esa información en la pantalla a través del método mostrar(). Dado que el método **mostrar()** no va a ser implementado en la clase Persona, será declarado como abstracto y no se incluirá su implementación:

```
protected abstract void mostrar();
```

Recuerda que el simple hecho de que la clase Persona contenga un método abstracto hace que sea clase sea abstracta (y deberá indicarse como tal en su declaración): public abstract class Persona. En el caso de la clase Alumno habrá que hacer una implementación específica del método mostrar y lo mismo para el caso de la clase Profesor.

1. Método mostrar para la clase Alumno.

```
// Redefinición del método abstracto mostrar en la clase Alumno
public void mostrar() {
 SimpleDateFormat formatoFecha = new  SimpleDateFormat("dd/MM/yyyy");
 String Stringfecha= formatoFecha.format(this.fechaNacim.getTime());
 System.out.printf("Nombre: %s\n", this.nombre);
 System.out.printf("Apellidos: %s\n", this.apellidos);
 System.out.printf("Fecha de nacimiento: %s\n", Stringfecha);
 System.out.printf("Grupo: %s\n", this.grupo);
 System.out.printf("Grupo: %5.2f\n", this.notaMedia);
}
```

2. Método mostrar para la clase Profesor.

```
// Redefinición del método abstracto mostrar en la clase Profesor
public void mostrar () {
 SimpleDateFormat formatoFecha = new SimpleDateFormat("dd/MM/yyyy");
 String Stringfecha= formatoFecha.format(this.fechaNacim.getTime());
 System.out.printf("Nombre: %s\n", this.nombre);
 System.out.printf("Apellidos: %s\n", this.apellidos);
 System.out.printf("Fecha de nacimiento: %s\n", Stringfecha);
 System.out.printf("Especialidad: %s\n", this.especialidad);
 System.out.printf("Salario: %7.2f euros\n", this.salario);
}
```

3. Programa de ejemplo de uso. Un pequeño programa de ejemplo de uso del método mostrar en estas dos clases podría ser:

```
// Declaración de objetos
Alumno alumno;
Profesor profe;
```

```
// Creación de objetos (llamada a constructores)
alumno= new Alumno ("Juan", "Torres", new GregorianCalendar (1990,
10, 6), "1DAM-B", 7.5);
profe= new Profesor ("Antonio", "Campos", new GregorianCalendar
(1970, 8, 15), "Mates", 2000);
// Utilización del método mostrar
alumno.mostrar();
profesor.mostrar();
```

Clases y métodos finales.

En unidades anteriores has visto el modificador **final**, aunque sólo lo has utilizado por ahora para atributos y variables (por ejemplo para declarar atributos constantes, que una vez que toman un valor ya no pueden ser modificados). Pero este modificador también puede ser utilizado con clases y con métodos (con un comportamiento que no es exactamente igual, aunque puede encontrarse cierta analogía: no se permite heredar o no se permite redefinir).

Una clase declarada como **final** no puede ser heredada, es decir, no puede tener clases derivadas. La jerarquía de clases a la que pertenece acaba en ella (no tendrá clases hijas):

[modificador_acceso] **final class** nombreClase [herencia] [interfaces]

Un método también puede ser declarado como final, en tal caso, ese método no podrá ser redefinido en una clase derivada:

[modificador_acceso] **final** <tipo> <nombreMetodo> ([parámetros]) [excepciones]

Si intentas redefinir un método final en una subclase se producirá un error de compilación.

Además de en la declaración de atributos, clases y métodos, el modificador **final** también podría aparecer acompañando a un método

de un parámetro. En tal caso no se podrá modificar el valor del parámetro dentro del código del método. Por ejemplo:

```
public final metodoEscribir(int p1, final int p2);
```

Dada la gran cantidad de contextos diferentes en los que se puede encontrar el modificador **final**, vale la pena hacer un repaso de todos los lugares donde puede aparecer y cuál sería su función en cada uno:

Distintos contextos en los que puede aparecer el modificador final	
Lugar	**Función**
Como modificador de clase.	La clase no puede tener subclases.
Como modificador de atributo.	El atributo no podrá ser modificado una vez que tome un valor. Sirve para definir constantes.
Como modificador al declarar un método	El método no podrá ser redefinido en una clase derivada.
Como modificador al declarar una variable referencia.	Una vez que la variable tome un valor referencia (un objeto), no se podrá cambiar. La variable siempre apuntará al mismo objeto, lo cual no quiere decir que ese objeto no pueda ser modificado internamente a través de sus métodos. Pero la variable no podrá apuntar a otro objeto diferente.
Como modificador en un parámetro de un método.	El valor del parámetro (ya sea un tipo primitivo o una referencia) no podrá modificarse dentro del código del método.

Figura 8: Resumen del uso de final.

Veamos un ejemplo de cada posibilidad:

1. Modificador de una clase.

```
public final class ClaseSinDescendencia { // Clase “no heredable”
   …
}
```

2. Modificador de un atributo.

```
public class ClaseEjemplo {
   // Valor constante conocido en tiempo de compilación
   final double PI= 3.14159265;
   // Valor constante conocido en tiempo de ejecución
```

```
        final int SEMILLA= (int) Math.random()*10+1;
        …
    }
```

3. Modificador de un método.

```
    public final metodo(int p1) { // Método "no redefinible"
        …
    }
```

4. Modificador en una variable referencia.

```
    // Referencia constante: siempre se apuntará al mismo objeto
    // Alumno recién creado, aunque este objeto pueda sufrir
    // modificaciones.
    final Alumno PRIMER_ALUMNO= new Alumno("Pepe", "Torres",9.55);
    // Si la variable no es una referencia (tipo primitivo), sería una
    // constante más (como un atributo constante).
    final int NUMERO_DIEZ = 10; // Valor constante
```

5. Modificador en un parámetro de un método.

```
    void metodoConParamFijos(final int p1, final int p2) {
        // p1 y p2 no podrán sufrir modificaciones aquí dentro
        …
    }
```

5.7. INTERFACES.

La herencia permite definir especializaciones (o extensiones) de una clase base que ya existe sin tener que volver a repetir de todo el código de ésta. Este mecanismo da la oportunidad de que la nueva clase especializada (o extendida) disponga de toda la interfaz que tiene su clase base.

También se ha comentado cómo los métodos abstractos permiten establecer un patrón que marca las líneas generales de un comportamiento común de superclase que deberían compartir de todas

las subclases.

Si llevamos al límite esta idea, podrías llegar a tener una clase abstracta donde todos sus métodos fueran abstractos. De este modo estarías dando únicamente el marco de comportamiento, sin ningún método implementado, de las posibles subclases que heredarán de esa clase abstracta.
La idea de una interfaz (o interface) es precisamente ésa: disponer de un mecanismo que permita especificar cuál debe ser el comportamiento que deben tener todos los objetos que formen parte de una determinada clasificación (no necesariamente jerárquica).

Una interfaz consiste principalmente en una lista de declaraciones de métodos sin implementar, que definen un comportamiento. Si se desea que una clase tenga ese comportamiento, tendrá que implementar esos métodos establecidos en la interfaz. En este caso **no se trata de una relación de herencia** (la clase A es una especialización de la clase B, o la subclase A "**es un**" B), sino más bien una relación "de implementación de comportamientos" (la clase A implementa los métodos establecidos en la interfaz B, o los comportamientos indicados por B son llevados a cabo por A; pero no significa que A sea un B, es "**A sabe hacer B**").

Imagina que estás diseñando una aplicación que trabaja con clases que representan distintos tipos de animales. Algunas de las acciones que quieres que lleven a cabo están relacionadas con el hecho de que algunos animales sean depredadores (por ejemplo: observar una presa, perseguirla, comérsela, etc.) o sean presas (observar, huir, esconderse, etc.). Si creas la clase León, esta clase podría implementar una interfaz Depredador, mientras que otras clases como Gacela implementarían las acciones de la interfaz Presa. Por otro lado,

podrías tener también el caso de la clase Rana, que implementaría las acciones de la interfaz Depredador (pues es cazador de pequeños insectos), pero también la de Presa (pues puede ser cazado y necesita las acciones necesarias para protegerse).

Concepto de interfaz.

Una interfaz en Java consiste esencialmente en una lista de declaraciones de métodos sin implementar, junto con un conjunto de constantes.

Estos métodos sin implementar indican un comportamiento, un tipo de conducta, aunque no especifican cómo será ese comportamiento (implementación), pues eso dependerá de las características específicas de cada clase que decida implementar esa interfaz. Podría decirse que una interfaz se encarga de establecer qué comportamientos hay que tener (qué métodos), pero no dice nada de cómo deben llevarse a cabo esos comportamientos (implementación). Se indica sólo la forma, no la implementación.

En cierto modo podrías imaginar el concepto de interfaz como un guión que dice: "éste es el protocolo de comunicación que deben presentar todas las clases que implementen esta interfaz". Se proporciona una lista de métodos públicos y, si quieres dotar a tu clase de esa interfaz, tendrás que definir todos y cada uno de esos métodos públicos.

En conclusión: una interfaz se encarga de establecer unas líneas generales sobre los comportamientos (métodos) que deberían tener los objetos de toda clase que implemente esa interfaz, es decir, que no indican lo que el objeto es (de eso se encarga la clase y sus superclases), sino acciones (capacidades) que el objeto debería ser capaz de realizar. Es por esto que el nombre de muchas interfaces en

Java termina con sufijos del tipo "-able", "-or", "-ente" y cosas del estilo, que significan algo así como capacidad o habilidad para hacer o ser receptores de algo (configurable, serializable, modificable, clonable, ejecutable, administrador, servidor, buscador, etc.), dando así la idea de que se tiene la capacidad de llevar a cabo el conjunto de acciones especificadas en la interfaz.

Imagina por ejemplo la clase Coche, subclase de Vehículo. Los coches son vehículos a motor, lo cual implica una serie de acciones como, por ejemplo, arrancar el motor o detener el motor. Esa acción no la puedes heredar de Vehículo, pues no todos los vehículos tienen porqué ser a motor (piensa por ejemplo en una clase Bicicleta), y no puedes heredar de otra clase pues ya heredas de Vehículo.

Una solución podría ser crear una interfaz Arrancable, que proporcione los métodos típicos de un objeto a motor (no necesariamente vehículos). De este modo la clase Coche sigue siendo subclase de Vehículo, pero también implementaría los comportamientos de la interfaz Arrancable, los cuales podrían ser también implementados por otras clases, hereden o no de Vehículo (por ejemplo una clase Motocicleta o bien una clase Motosierra). La clase Coche implementará su método arrancar de una manera, la clase Motocicleta lo hará de otra (aunque bastante parecida) y la clase Motosierra de otra forma probablemente muy diferente, pero todos tendrán su propia versión del método arrancar() como parte de la interfaz Arrancable.

Según esta concepción, podrías hacerte la siguiente pregunta: ¿podrá una clase implementar varias interfaces? La respuesta en este caso sí es afirmativa. Una clase puede adoptar distintos modelos de comportamiento establecidos en diferentes interfaces. Es decir una

clase puede implementar varias interfaces.

Definición de interfaces.

La declaración de una interfaz en Java es similar a la declaración de una clase, aunque con algunas variaciones:

- Se utiliza la palabra reservada **interface** en lugar de **class**.
- Puede utilizarse el modificador **public**. Si incluye este modificador la interfaz debe tener el mismo nombre que el archivo .java en el que se encuentra (exactamente igual que sucedía con las clases). Si no se indica el modificador **public**, el acceso será "de paquete" (como con las clases).
- Todos los miembros de la interfaz (atributos y métodos) son public de manera implícita. No es necesario indicar el modificador **public**, aunque puede hacerse.
- Todos los atributos son de tipo **final** y **public** (tampoco es necesario especificarlo), es decir, constantes y públicos. Hay que darles un valor inicial.
- Todos los métodos son abstractos también de manera implícita (tampoco hay que indicarlo). No tienen cuerpo, tan solo la cabecera.

Como puedes observar, una interfaz consiste esencialmente en una lista de atributos finales (constantes) y métodos abstractos (sin implementar). Su sintaxis quedaría entonces:

```
[public] interface NombreInterfaz {
[public] [final] tipo atributo1= <valor1>;
[public] [final] tipo atributo2= <valor2>;
...
[public] [abstract] tipo nombreMetodo1([parámetros]);
[public] [abstract] tipo nombreMetodo2([parámetros]);
```

```
...
}
```

Si te fijas, la declaración de los métodos termina en punto y coma, pues no tienen cuerpo, al igual que sucede con los métodos abstractos de las clases abstractas. El ejemplo de la interfaz Depredador que hemos visto antes podría quedar entonces así:

```
public interface Depredador {
   void localizar(Animal presa);
   void cazar(Animal presa);
   ...
}
```

Serán las clases que implementen esta interfaz (León, Leopardo, Cocodrilo, Rana, Lagarto, Hombre, etc.) las que definan cada uno de los métodos por dentro.

Los métodos de una interfaz en Java tienen que ser obligatoriamente declarados como **public** y **abstract**. Si no se indica así, se producirá un error de compilación. ¿Verdadero o Falso? Verdadero Falso

No es necesario hacerlo. Todos los métodos de una interfaz serán siempre public y abstract aunque no se indique. El compilador lo considerará así de forma implícita. Lo que sí produciría un error de compilación sería la colocación de un modificador de acceso diferente a public (por ejemplo private o protected) o el empleo del modificador static. Pero si no se indica nada, el método será public y abstract.

EJEMPLO: Crea una interfaz en Java cuyo nombre sea **Imprimible** y que contenga algunos métodos útiles para mostrar el contenido de una clase:

1. Método **devolverContenidoString**, que crea un **String** con una representación de todo el contenido público (o que se decida

que deba ser mostrado) del objeto y lo devuelve. El formato será una lista de pares "nombre=valor" de cada atributo separado por comas y la lista completa encerrada entre llaves:

```
"{ atributo1 = valor1, ..., atributon = valorn}".
```

2. Método **devolverContenidoArrayList**, que crea un **ArrayList** de String con una representación de todo el contenido público (o que se decida que deba ser mostrado) del objeto y lo devuelve.
3. Método **devolverContenidoHashtable**, similar al anterior, pero en lugar devolver en un ArrayList los valores de los atributos, se devuelve en una Hashtable en forma de pares (nombre, valor). Se trata simplemente de declarar la interfaz e incluir en su interior esos tres métodos:

```
public interface Imprimible {
   String    devolverContenidoString();
   ArrayList devolverContenidoArrayList();
   HashTable devolverContenidoHashtable();
}
```

El cómo se implementarán cada uno de esos métodos dependerá exclusivamente de lo que cada clase decida.

Implementación de interfaces.

Como ya has visto, todas las clases que implementan una determinada interfaz están obligadas a proporcionar una definición (implementación) de los métodos de esa interfaz, adoptando el modelo de comportamiento propuesto por ésta.

Dada una interfaz, cualquier clase puede especificar dicha interfaz mediante el mecanismo denominado implementación de interfaces. Para ello se utiliza la palabra reservada implements:

class NombreClase **implements** NombreInterfaz { ...

De esta manera, la clase está diciendo algo así como "la interfaz indica los métodos que debo implementar, pero voy a ser yo (la clase) quien los implemente".

Es posible indicar varios nombres de interfaces separándolos por comas:

class NombreClase **implements** Interfaz1, Interfaz2,... {

Cuando una clase implementa una interfaz, tiene que redefinir sus métodos nuevamente con acceso público. Con otro tipo de acceso se producirá un error de compilación. Es decir, que del mismo modo que no se podían restringir permisos de acceso en la herencia de clases, tampoco se puede hacer en la implementación de interfaces.

Una vez implementada una interfaz en una clase, los métodos de esa interfaz tienen exactamente el mismo tratamiento que cualquier otro método, sin ninguna diferencia, pudiendo ser invocados, heredados, redefinidos, etc.

En el ejemplo de los depredadores, al definir la clase León, habría que indicar que implementa la interfaz Depredador:

```
class Leon implements Depredador {...
```

Y en su interior habría que implementar aquellos métodos que contenga la interfaz:

```
void localizar(Animal presa) {
   // Implementación del método localizar para un león
   ...
}
```

En el caso de clases que pudieran ser a la vez Depredador y Presa,

tendrían que implementar ambas interfaces, como podría suceder con la clase Rana:

```
class Rana implements Depredador, Presa {
```

Y en su interior habría que implementar aquellos métodos que contengan ambas interfaces, tanto las de Depredador (localizar, cazar, etc.) como las de Presa (observar, huir, etc.).

EJEMPLO: Haz que las clases Alumno y Profesor implementen la interfaz Imprimible que se ha escrito en el ejercicio anterior.

La primera opción que se te puede ocurrir es pensar que en ambas clases habrá que indicar que implementan la interfaz Imprimible y por tanto definir los métodos que ésta incluye: devolverContenidoString, devolverContenidoHashtable y devolverContenidoArrayList.

Si las clases Alumno y Profesor no heredaran de la misma clase habría que hacerlo obligatoriamente así, pues no comparten superclase y precisamente para eso sirven las interfaces: para implementar determinados comportamientos que no pertenecen a la estructura jerárquica de herencia en la que se encuentra una clase (de esta manera, clases que no tienen ninguna relación de herencia podrían compartir interfaz).

Pero en este caso podríamos aprovechar que ambas clases sí son subclases de una misma superclase (heredan de la misma) y hacer que la interfaz Imprimible sea implementada directamente por la superclase (Persona) y de este modo ahorrarnos bastante código. Así no haría falta indicar explícitamente que Alumno y Profesor implementan la interfaz Imprimible, pues lo estarán haciendo de forma implícita al heredar de una clase que ya ha implementado esa interfaz (la clase Persona, que es padre de ambas).

Una vez que los métodos de la interfaz estén implementados en la clase Persona, tan solo habrá que redefinir o ampliar los métodos de la interfaz para que se adapten a cada clase hija específica (Alumno o Profesor), ahorrándonos tener que escribir varias veces la parte de código que obtiene los atributos genéricos de la clase Persona.

1. Clase Persona. Indicamos que se va a implementar la interfaz Imprimible:

```
public abstract class Persona implements Imprimible { ...
```

Definimos el método **devolverContenidoHashtable** a la manera de como debe ser implementado para la clase Persona. Podría quedar, por ejemplo, así:

```
public Hashtable devolverContenidoHashtable () {
   Hashtable contenido= new Hashtable(); // Creamos la Hashtable
   // Añadimos los atributos de la clase
   SimpleDateFormat fFecha = new SimpleDateFormat("dd/MM/yyyy");
   String sFecha= fFecha.format(this.fechaNacim.getTime());
   contenido.put ("nombre", this.nombre);
   contenido.put ("apellidos", this.apellidos);
   contenido.put ("fechaNacim", stringFecha);
   return contenido; // Devolvemos la Hashtable
}
```

Del mismo modo, definimos también el método devolverContenidoArrayList:

```
public ArrayList devolverContenidoArrayList () { ... }
```

Y por último el método devolverContenidoString:

```
public String devolverContenidoString () { ... }
```

2. Clase Alumno. Esta clase hereda de la clase Persona, de manera que

heredará los tres métodos anteriores. Tan solo habrá que redefinirlos para que, aprovechando el código ya escrito en la superclase, se añada la funcionalidad específica que aporta esta subclase.

```
public class Alumno extends Persona { ...
```

Como puedes observar no ha sido necesario incluir el **implements** Imprimible, pues el **extends** Persona lo lleva implícito dado que Persona ya implementaba ese interfaz. Lo que haremos entonces será llamar al método que estamos redefiniendo utilizando la referencia a la superclase **super**.

El método devolverContenidoHashtable podría quedar así:

```
public Hashtable devolverContenidoHashtable() {
   // Llamada al método de la superclase
   Hashtable contenido= super.devolverContenidoHashtable() ;
   // Añadimos los atributos específicos de la clase
   contenido.put("grupo", this.salario);
   contenido.put("notaMedia", this.especialidad);
   // Devolvemos la Hashtable rellena
   return contenido;
}
```

3. Clase Profesor. En este caso habría que proceder exactamente de la misma manera que con la clase Alumno: redefiniendo los métodos de la interfaz **Imprimible** para añadir la funcionalidad específica que aporta.

Herencia de interfaces.

Las interfaces, al igual que las clases, también permiten la herencia. Para indicar que una interfaz hereda de otra se indica nuevamente con la palabra reservada **extends**. Pero en este caso sí se permite la herencia múltiple de interfaces. Si se hereda de más de una interfaz se indica con la lista de interfaces separadas por comas. Por ejemplo,

dadas las interfaces InterfazUno e InterfazDos:

```
public interface InterfazUno {
   // Métodos y constantes de la interfaz Uno
}
public interface InterfazDos {
   // Métodos y constantes de la interfaz Dos
}
```

Podría definirse una nueva interfaz que heredara de ambas:

```
public interface InterfazCompleja extends InterfazUno, InterfazDos{
  // Métodos y constantes de la interfaz compleja
}
```

EJEMPLO: Localiza en la API de Java algún ejemplo de interfaz que herede de una o varias interfaces (puedes consultar la documentación de referencia de la API de Java).

Existen una gran cantidad de interfaces en la API de Java que heredan de otras interfaces. Aquí tienes un par de ejemplos:

- La interfaz java.awt.event.ActionListener, que hereda de java.util.EventListener.
- La interfaz org.omg.CORBA.Policy, que hereda de org.omg.CORBA.PolicyOperations, org.omg.CORBA.Object y org.omg.CORBOA.portable.IDLEntity.

5.7.1. ¿QUÉ USO, CLASE ABSTRACTA O INTERFACE?

Observando el concepto de interfaz, podría caerse en la tentación de pensar que es prácticamente lo mismo que una clase abstracta en la que todos sus métodos sean abstractos.

Es cierto que en ese sentido existe un gran parecido formal entre una clase abstracta y una interfaz, pudiéndose en ocasiones utilizar indistintamente una u otra para obtener un mismo fin. Pero, a pesar

de ese gran parecido, existen algunas diferencias, no sólo formales, sino también conceptuales, muy importantes:

- Una clase no puede heredar de varias clases, aunque sean abstractas (herencia múltiple). Sin embargo sí puede implementar una o varias interfaces y además seguir heredando de una clase.
- Una interfaz no puede definir métodos (no implementa su contenido), tan solo los declara o enumera.
- Una interfaz puede hacer que dos clases tengan un mismo comportamiento independientemente de sus ubicaciones en una determinada jerarquía de clases (no tienen que heredar las dos de una misma superclase, pues no siempre es posible según la naturaleza y propiedades de cada clase).
- Una interfaz permite establecer un comportamiento de clase sin apenas dar detalles, pues esos detalles aún no son conocidos (dependerán del modo en que cada clase decida implementar la interfaz).
- Las interfaces tienen su propia jerarquía, diferente e independiente de la jerarquía de clases.

De todo esto puede deducirse que una clase abstracta proporciona una interfaz disponible sólo a través de la herencia. Sólo quien herede de esa clase abstracta dispondrá de esa interfaz. Si una clase no pertenece a esa misma jerarquía (no hereda de ella) no podrá tener esa interfaz. Eso significa que para poder disponer de la interfaz podrías:

1. Volver a escribirla para esa jerarquía de clases. Lo cual no parece una buena solución.
2. Hacer que la clase herede de la superclase que proporciona la

interfaz que te interesa, sacándola de su jerarquía original y convirtiéndola en clase derivada de algo de lo que conceptualmente no debería ser una subclase. Es decir, estarías forzando una relación "es un" cuando en realidad lo más probable es que esa relación no exista. Tampoco parece la mejor forma de resolver el problema.

Sin embargo, una interfaz sí puede ser implementada por cualquier clase, permitiendo que clases que no tengan ninguna relación entre sí (pertenecen a distintas jerarquías) puedan compartir un determinado comportamiento (una interfaz) sin tener que forzar una relación de herencia que no existe entre ellas.

A partir de ahora podemos hablar de otra posible relación entre clases: la de compartir un determinado comportamiento (interfaz). Dos clases podrían tener en común un determinado conjunto de comportamientos sin que necesariamente exista una relación jerárquica entre ellas. Tan solo cuando haya realmente una relación de tipo "es un" se producirá herencia.

Si sólo vas a proporcionar una lista de métodos abstractos (interfaz), sin definiciones de métodos ni atributos de objeto, suele ser recomendable definir una interfaz antes que clase abstracta. Es más, cuando vayas a definir una supuesta clase base, puedes comenzar declarándola como interfaz y sólo cuando veas que necesitas definir métodos o variables miembro, puedes entonces convertirla en clase abstracta (no instanciable) o incluso en una clase instanciable.

Conceptualmente SER o PODER, esa es la diferencia:

- **Clases Abstractas**: indica que una clase está incompleta y que sólo se va a utilizar como una clase base. No se podrá definir un

nuevo objeto directamente de ella. Todas las clases que hereden de esta clase abstracta la toman como base su comportamiento, formando parte de su esqueleto, lo podríamos definir como **subclase «ES UN(A)» superclase**.

- **Interfaces**: una serie de métodos, propiedades, índices y eventos, que podemos definir, y que la clase que herede de ellos debe cumplir. Esto significa que clases que implementen esta interfaz pueden ser diferentes casi por completo, pero contener todo lo que la interfaz define. Por esto mismo, podemos decir que es un **clase «PUEDE HACER» interfaz**.

EJEMPLO: Queremos definir una serie de animales, un pato, un tiburón, un gorrión y un mono. Si lo pensamos, todos SON animales, por lo tanto esta será nuestra clase abstracta.

```
public abstract class Animal {
    private String nombre;
    public Animal(String n) { nombre = n;  }
    public getNombre(){ return nombre; }
    public abstract void comer(String comida, int cantidad);
}
```

El pato y el gorrión, a su vez, PUEDEN volar y el tiburón y el pato PUEDEN nadar. Éstas serán nuestras interfaces.

```
public interface Volar {  void vuela();  }

public interface Nadar { void nada(); }
```

Este sería el resultado:

```
public class Pato extends Animal implements Volar, Nadar {
    public Pato() { super("pato"); }
    public void comer(String comida, int cantidad);
    public void vuela() { /*...*/ }
    public void nada() { /*...*/ }
```

```
}

public class Tiburon extends Animal implements Nadar {
    public Tiburon() { super("tiburon"); }
    public void come(String comida, int cantidad){ /*...*/ };
    public void nada() { /*...*/ };
}

public class Gorrion extends Animal implements Volar {
    public Gorrion() { super("gorrion"); }
    public void comer(String comida, int cantidad){ /*...*/  };
    public void vuela() { /*...*/ }
}

public class Mono extends Animal {
    public Mono() { super("mono"); }
    public void come(String comida, int cantidad){ /*...*/  };
 }
```

5.7.2. ALGUNAS INTERFACES de JAVA.

El trabajo con interfaces es algo habitual en el desarrollo de aplicaciones en Java. Es por tanto muy importante comprender correctamente su funcionamiento y la interacción con las distintas bibliotecas (paquetes de clases e interfaces) que proporcionan las APIs. Estas clases e interfaces son fundamentales para la creación de aplicaciones y tendrás que utilizarlas en multitud de ocasiones (además, por supuesto, de las que tengas que desarrollar por ti mismo).

Vamos a ver varios ejemplo de una interfaz proporcionada por la API de Java que puede ser implementada por alguna clase creada por ti dentro de una pequeña aplicación. Hemos escogido la interfaz **ActionListener**.

ACTIONLISTENER

Las clases que quieran realizar una determinada acción cada vez que se produzca cierto evento en el sistema deben implementar esta interfaz.

Este tipo de interfaces se encuentran dentro de los **Event Listeners** u "oyentes de eventos" y son útiles para detectar que se ha producido un determinado evento asíncrono durante la ejecución de tu aplicación (pulsación de una tecla, clic de ratón, etc.). Son intensivamente utilizadas en el desarrollo de las GUI.

Es interesante que estés al día sobre los conceptos del modelo de gestión de eventos de las GUI: evento, oyente, gestión de eventos, etc. Si deseas profundizar algo más en el tema de los Event Listeners puedes echar un vistazo a los tutoriales de iniciación de Java en el sitio web de Oracle (en inglés) en el que se explican detalladamente ejemplos sencillos así como otros bastante más complejos: enlace.

Vamos a ver un ejemplo lo más sencillo posible: se trata de desarrollar una pequeña aplicación de escritorio (utilizando la API de Swing) con una ventana que contenga un par de botones y que al pulsar alguno de esos botones (evento), la aplicación sea capaz de detectar que se ha producido ese evento y por tanto realizar una determinada acción (por ejemplo generar un sonido o mostrar un determinado texto).

Para ello es necesario que la clase que vaya a gestionar el evento sea un "oyente" de ese evento, es decir, que sea capaz de enterarse de que se ha producido ese evento. Esa capacidad o habilidad la proporciona la API de Java a través de las interfaces de tipo "oyente". En concreto vamos a implementar la interfaz ActionListener.

La interfaz **ActionListener** tiene un único método: **actionPerformed**. Toda clase W que implemente esta interfaz tendrá que definir ese método. Si incrustamos un determinado objeto gráfico G en una ventana W y asociamos a ese objeto gráfico la clase W, indicando que es su oyente y gestor de eventos, cuando se produzca

algún evento sobre G se ejecutará el método **actionPerformed** implementado por W.

En nuestro ejemplo podríamos hacer lo siguiente:

- Definir una ventana principal, por ejemplo una clase basada en el tipo marco o frame (subclase de **JFrame**) que implemente la interfaz **ActionListener**.
- Definir uno o varios botones (objetos de la clase **JButton**) dentro de la ventana.
- Asociar como oyente a los objetos de tipo botón la propia ventana principal (frame), pues va a ser capaz de gestionar eventos (implementa el método **actionPerformed**).
- Implementar el método actionPerformed como un método de la ventana principal (frame) para reaccionar de algún modo cuando se produzca algún evento desencadenado por un botón (pues se ha establecido a la ventana principal como oyente y gestora de los eventos del botón).

Ejemplo de implementación de la interfaz ActionListener

Creamos un nuevo proyecto Java de tipo "Aplicación de escritorio Java", creamos una clase principal que herede de JFrame e incluimos las bibliotecas de la intefaz gráfica	`import java.awt.*`
	`import javax.swing.JFrame;`
	
	`public class Ejemplo extends JFrame {`
	`  public static void main(String[] args) {`
	`  }`
	`}`

Incrustamos algunos	`import java.awt.*` `import javax.swing.JFrame;`

componentes gráficos en el frame que acabamos de crear	`public class Ejemplo extends JFrame {`
	`JPanel p;`
	`JButton b1, b2;`
	`public Ejemplo() {`
	`b1 = new JButton("Un pitido");`
	`b2 = new JButton("Dos pitidos");`
	`p = new JPanel();`
	`p.add(b1);`
	`p.add(b2);`
	`this.getContentPane().add(p);`
	`}`
	`}`

Asignamos oyentes a los componentes gráficos	`import java.awt.*`
	`import javax.swing.JFrame;`
	
	`public class Ejemplo extends JFrame implements ActionListener{`
	`JPanel p;`
	`JButton b1, b2;`
	`public Ejemplo(String[] args) {`
	`b1 = new JButton("Un pitido");`
	`b2 = new JButton("Dos pitidos");`
	`p = new JPanel();`
	`p.add(b1);`
	`p.add(b2);`
	`this.getContentPane().add(p);`
	`b1.addActionListener(this);`
	`b2.addActionListener(this);`
	`}`
	`}`

Implementamos el método actionPerformed	`// ActionPerformed es necesario para la`
	`// interfaz ActionListener`
	`public void ActionPerformed(ActionEvent e) {`
	`// Averigura botón pulsado`
	`JButton pulsado = (JButton)e.getSource();`
	`if( pulsado == b1 ) {`
	`Toolkit.getdefaultToolkit().beep();`
	`} else if (pulsado == b2) {`
	`Toolkit.getdefaultToolkit().beep();`
	`Toolkit.getdefaultToolkit().beep();`
	`}`

	`}`

Implementamos el método main	`public static void main(String[] args) {`
	`  Ejemplo ventana = new Ejemplo();`
	`  ventana.setTitle("Action Listener");`
	`  ventana.setSize(300,80);`
	`  ventana.setVisible(true);`
	`}`

Algunas otras interfaces comunes de la API de Java

Utilizarás mucho las interfaces cuando desarrolles aplicaciones de Java. La API de Java contiene numerosas interfaces y muchos de los métodos de la API de Java reciben argumentos de interfaces y devuelven valores interfaces. **Debes conocer como mínimo estas:**

- **Comparable** Java contiene varios operadores de comparación (<, <=, >, >=, ==, ! =) que permiten comparar valores primitivos. Sin embargo, estos operadores no se pueden utilizar para comparar objetos. La interfaz **Comparable** se utiliza para permitir que los objetos de una clase que implementa la interfaz se comparen entre sí. La interfaz Comparable se utiliza a menudo para ordenar objetos en una colección, como arrays.
- **Serializable** Una interfaz que se utiliza para identificar clases cuyos objetos pueden escribirse en (serializarse), o leerse desde (deserializarse) algún tipo de almacenamiento (archivo en disco, campo de base de datos, string) o transmitirse a través de una red.
- **Runnable**, La implementa cualquier clase que represente una tarea a realizar. Sus objetos con frecuencia se ejecutan en paralelo, usando una técnica llamada multihilos. La interfaz contiene un método, **run()**, que describe el comportamiento de

un objeto al ejecutarse.

- **Interfaces de escucha de eventos de las GUI**: en una GUI, por ejemplo, en su navegador Web, podrías escribir en un campo de texto la dirección de un sitio Web para visitarlo, o podrías hacer clic en un botón para regresar al sitio anterior. El navegador Web responde a su interacción y realiza una tarea. La interacción se conoce como **un evento** y el código que utiliza el navegador para responder a un evento se conoce como **manejador de eventos**. Éstos se declaran en clases que implementan una interfaz de escucha de eventos apropiada. Cada interfaz de escucha de eventos especifica uno o más métodos que deben implementarse para responder a las interacciones de los usuarios.
- **AutoCloseable** Es implementada por clases que pueden usarse con la instrucción **try** con recursos para ayudar a evitar fugas de recursos.

5.7.3. HERENCIA MÚLTIPLE CON INTERFACES.

Una interfaz no tiene espacio de almacenamiento asociado (no se van a crear objetos de un tipo de interfaz), es decir, no tiene implementación.

En algunas ocasiones es posible que interese representar la situación de que "una clase X es de tipo A, de tipo B y de tipo C", siendo A, B, C clases disjuntas (no heredan unas de otras). Hemos visto que sería un caso de herencia múltiple que Java no permite.

Para poder simular algo así, podrías definir tres interfaces A, B, C que indiquen los comportamientos (métodos) que se deberían tener según se pertenezca a una supuesta clase A, B, o C, pero sin implementar

ningún método concreto ni atributos de objeto (sólo interfaz).

De esta manera la clase X podría a la vez:

1. Implementar las interfaces A, B, C, que la dotarían de los comportamientos que deseaba heredar de las clases A, B, C.
2. Heredar de otra clase Y, que le proporcionaría determinadas características dentro de su jerarquía de objeto (atributos, métodos implementados y métodos abstractos).

En el ejemplo que hemos visto de las interfaces Depredador y Presa, tendrías un ejemplo de esto: la clase Rana, que es subclase de Anfibio, implementa una serie de comportamientos propios de un Depredador y a la vez, otros más propios de una Presa. Esos comportamientos (métodos) no forman parte de la superclase Anfibio, sino de las interfaces. Si se decide que la clase Rana debe de llevar a cabo algunos otros comportamientos adicionales, podrían añadirse a una nueva interfaz y la clase Rana implementaría una tercera interfaz.

De este modo, con el mecanismo "una herencia pero varias interfaces", podrían conseguirse resultados similares a los obtenidos con la herencia múltiple.

Ahora bien, del mismo modo que sucedía con la herencia múltiple, **puede darse el problema de la colisión de nombres al implementar dos interfaces que tengan un método con el mismo identificador**. En tal caso puede suceder lo siguiente:

- Si los dos métodos tienen diferentes parámetros no habrá problema aunque tengan el mismo nombre pues se realiza una sobrecarga de métodos.
- Si los dos métodos tienen un valor de retorno de un tipo

diferente, se producirá un error de compilación (al igual que sucede en la sobrecarga cuando la única diferencia entre dos métodos es ésa).

- Si los dos métodos son exactamente iguales en identificador, parámetros y tipo devuelto, entonces solamente se podrá implementar uno de los dos métodos. En realidad se trata de un solo método pues ambos tienen la misma interfaz (mismo identificador, mismos parámetros y mismo tipo devuelto).

La utilización de nombres idénticos en diferentes interfaces que pueden ser implementadas a la vez por una misma clase puede causar, además del problema de la colisión de nombres, dificultades de legibilidad en el código, pudiendo dar lugar a confusiones. Si es posible intenta evitar que se produzcan este tipo de situaciones.

EJEMPLO: Localiza en la API de Java algún ejemplo de clase que implemente varias interfaces diferentes (puedes consultar la documentación de referencia de la API de Java).

Existen una gran cantidad de clases en la API de Java que implementan múltiples interfaces. Aquí tienes un par de ejemplos:

- La clase **javax.swing.JFrame**, que implementa las interfaces WindowConstants, Accessible y RootPaneContainer.
- La clase **java.awt.Component**, que implementa las interfaces ImageObserver, MenuContainer y Serializable.

5.8. MEJORAS A PARTIR DE JAVA 8.

Métodos default de una interfaz

Antes de Java SE 8, los métodos de las interfaces solamente podían

ser métodos **public abstract**. Esto significa que una interfaz especificaba qué operaciones debía realizar una clase implementadora, pero no cómo debía realizarlas.

En Java SE 8 las interfaces también pueden contener métodos **default public** con implementaciones predeterminadas concretas que especifiquen cómo deben realizarse las operaciones cuando una clase implementadora no sobrescriba los métodos. Si una clase implementa dicha interfaz, la clase también recibe las implementaciones **default** de esa interfaz (si las hay). Para declarar un método **default**, escribes la palabra clave **default** antes del tipo de valor de retorno del método y proporcionas una implementación concreta del mismo.

Los métodos **default** son automáticamente **public** (no hay que indicarlo). Ejemplo:

```
public interface Leible {     // representa un origen de datos
   public char leeChar();     // lee un char
   default public String leeLinea() {     // lee hasta salto
      StringBuilder linea = new StringBuilder();
      char ch = leeChar();
      while (ch != '\n') {
         linea.append(ch);
         ch = leeChar();
      }
      return linea.toString();
   }
}
```

Una clase concreta que implemente esta interfaz debe aportar una implementación para el método **leeChar()**. Hereda la definición de **leeLinea()** de la interfaz, aunque puede implementar una nueva definición si es necesario. La referencia a **leeChar()** en la definición es polimórfica. Una clase con main() que implementa la interfaz:

```
public class Stars implements Leible {
```

```
        public char leeChar() {
            if (Math.random() > 0.02)
                return '*';
            else
                return '\n';
        }
        public static void main(String[] args) {
            Stars stars = new Stars();
            for (int i = 0 ; i < 10; i++ ) {
                String linea = stars.leeLinea();
                System.out.println( linea );
            }
        }
    }
```

Agregar métodos a las interfaces existentes

Antes de Java SE 8, agregar métodos a una interfaz descompondría a las clases implementadoras que no implementaran los nuevos métodos. Recuerde que, si no implementó cada uno de los métodos de la interfaz, tiene que declarar su clase como **abstract**.

Cualquier clase que implemente la interfaz original no se descompondrá cuando se agregue un método **default**. La clase simplemente recibirá el nuevo método **default**. Cuando una clase implementa una interfaz de Java SE 8, la clase "firma un contrato" con el compilador que dice: "Declararé todos los métodos abstract especificados por la interfaz o declararé mi clase como abstract". La clase implementadora no tiene que sobrescribir los métodos **default** de la interfaz, pero puede hacerlo si es necesario.

> **Nota**: *Los métodos **default** de Java SE 8 permiten evolucionar a las interfaces existentes al agregar nuevos métodos a esas interfaces sin descomponer el código que las usa.*

> **Nota**: *Antes de Java SE 8, se utilizaba por lo general una*

interfaz (en vez de una clase abstract) cuando no había detalles de implementación a heredar, es decir, no había campos ni implementaciones de métodos. Con los métodos default, puede declarar implementaciones de métodos comunes en las interfaces, lo que le da más flexibilidad al diseñar sus clases.

Métodos static de una interfaz

Antes de Java SE 8, era común asociar con una interfaz a una clase que tuviera métodos ayudantes **static** para trabajar con objetos que implementaran la interfaz. Por ejemplo la clase **Collections**, que contiene muchos **métodos ayudantes static** para trabajar con objetos que implementan las interfaces **Collection**, **List**, **Set** y demás. Por ejemplo, el método **sort()** de Collections puede ordenar objetos de cualquier clase que implemente a la interfaz **List**. Con los métodos static de una interfaz, dichos métodos ayudantes pueden ahora declararse directamente en las interfaces, en vez de hacerlo en cada una de las clases ayudantes.

Interfaces funcionales

A partir de Java SE 8, cualquier interfaz que contenga sólo un método **abstract** se conoce como interfaz funcional. Hay muchas de esas interfaces en las API de Java 7 y muchas nuevas en Java 8.

Algunas de las interfaces funcionales que usarás a menudo son:

- **ActionListener**: podrás implementar esta interfaz para definir un método que se invoca cuando el usuario hace clic en un botón.
- **Comparator**: para definir un método que puede comparar dos objetos de un tipo dado para determinar si el primer objeto es menor, igual o mayor que el segundo.

- **Runnable**: para definir una tarea que pueda ejecutarse en paralelo con otras partes de su programa.

Las interfaces funcionales se usan mucho con las nuevas capacidades lambda de Java SE 8. También se implementa frecuentemente una interfaz mediante la creación de lo que se conoce como **una clase interna anónima**, la cual implementa el o los métodos de la interfaz. En Java SE 8, las lambdas dan una notación abreviada para crear métodos anónimos que el compilador traduce de manera automática en clases internas anónimas.

5.8.1. CLASES ANIDADAS.

Una clase es un bloque de alto nivel de un programa, representando una idea compleja: sus datos asociados y su comportamiento. Sin embargo es engorroso tener que escribir clases muy sencillas, es más el trabajo de hacerlas que el beneficio que aportan (un fichero para ellas, etc.).

Esto en Java puede solucionarse con la anidación de unas clases dentro de otra clase. La pequeña clase trivial puede pertenecer a una clase más compleja. Esto se utiliza mucho sobre todo cuando la clase sencilla solamente sirve para facilitar el trabajo de la mayor.

En Java, **una clase anidada es cualquier clase cuya definición está dentro de otra clase** (de hecho, una clase puede estar anidada hasta dentro de una subrutina, que a su vez, está dentro de una clase). Las clases anidadas pueden tener nombre o ser anónimas. Y las que tienen nombre pueden ser estáticas o no estáticas.

Las clases internas se utilizan en algunos casos para:

- Agrupar clases que sólo tiene sentido que existan en el entorno de la clase en la que han sido definidas, de manera que se oculta

su existencia al resto del código.

- Incrementar el nivel de encapsulación y ocultamiento.
- Proporcionar un código fuente más legible y fácil de mantener (el código de las clases internas y anidadas está más cerca de donde es usado).

Se pueden distinguir varios tipos de clases internas en Java:

- Clases **internas estáticas** (conocidas como **clases anidadas**), declaradas con el modificador **static**. En el caso de que la interna sea una interface, aunque no uses la palabra static, lo es de forma implícita.
- **Clases internas miembro** (conocidas como **clases internas)**, declaradas a nivel de la clase contenedora como no estática. Las clases extenas/internas (no estáticas) tienen una relación especial porque pueden acceder mutuamente a sus miembros independientemente de los modificadores de acceso. Una limitación de las clases internas es que **no pueden tener miembros estáticos**.
- **Clases internas locales**, que se declaran en el interior de un bloque de código (normalmente dentro de un método).
- **Clases anónimas**, similares a las internas locales, pero sin nombre (sólo existirá un objeto de ellas y, al no tener nombre, no tendrán constructores). Se suelen usar en la gestión de eventos en los interfaces gráficos.

CLASE ANIDADA STATIC

Las clases anidadas, como miembros de una clase que son (miembros de

claseExterna), pueden ser declaradas con los modificadores **public**, **protected**, **private** o de paquete, como el resto de miembros.

La definición es como la de cualquier otra clase, salvo porque se produce dentro de otra clase y tiene el modificador **static** como parte de su declaración.

Es un elemento estático de la clase que la contiene. Se usa para crear objetos y si se utiliza desde fuera de la clase donde se define, su llamada debe prefijarse con la clase donde está definida (clase_contenedora.clase_anidada).

Por ejemplo, supongamos una clase llamada ModeloAlambre representa un conjunto de líneas en el espacio 3D (se usan para representar figuras 3D). Imagina que esta clase tiene una clase estática anidada llamada Linea que representa una línea. Fuera de la clase ModeloAlambre, esa clase se referencia como ModeloAlambre.Linea. Ejemplo:

```
public class ModeloAlambre {
   // Otros miembros de la clase...
   static public class Linea {
      // Una linea 3D desde el pto (x1,y1,z1) hasta (x2,y2,z2)
      double x1, y1, z1;
      double x2, y2, z2;
   } // fin clase Linea
   // Otros miembros de la clase ModeloAlambre...
} // fin clase ModeloAlambre
```

Así que dentro de la clase podemos crear objetos Linea con la sentencia **new Linea()** y desde fuera con **new ModeloAlambre.Linea()**.

- **Una clase estática anidada puede acceder a todos los miembros de la clase contenedora (incluso a los privados)**.
- **De igual forma, la clase contenedora accede a todos los**

miembros de la clase anidada (incluso los privados).

- **Si la propia clase anidada se declara como private, solo podrá utilizarse dentro de la clase contenedora, no desde el exterior**.

Cuando compilas el fichero fuente anterior, se generan dos ficheros .class. El nombre del fichero de la clase Linea será **ModeloAlambre$Linea.class**.

CLASES ANIDADAS NO STATIC (Inner Classes)

Las clases anidadas no estáticas se denominan **clases internas**. Se diferencian de las estáticas en que están **asociadas a los objetos, no a la clase**.

Cada objeto tendrá su propia clase y tendrá acceso a todos los métodos y variables de instancia del objeto, aunque sean privados. La regla para decidir cuando hacer una clase anidada estática o no es sencilla: **si necesita acceder a las variables de instancia/métodos de los objetos, se hace interna, en otro caso estática**.

En la mayoría de los casos, una clase interna se usa solamente dentro de la clase donde se define. Desde dentro de la clase contenedora puedes crear variables y declarar objetos usando el nombre de la clase. Pero desde fuera, debes prefijar con el objeto: objeto.clase_anidada (cuando trabajas dentro de la clase se usa this de forma implícita). Ejemplo:

```
public class Poker {     // Juego de poker
   class Jugador {       // Un jugador
      //...
   } // fin clase Jugador
   private Mazo m;      // El mazo de cartas
   private int bote;    // cantidaad de dinero en juego
```

```
    //...
} // fin clase Poker
```

Si juego es una variable de tipo Poker, juego tiene su propia copia de la clase Jugador. Dentro de un método de instancia (del objeto) se puede crear un nuevo jugador haciendo la llamada **new Jugador()** y desde fuera del objeto se haría **juego.new Jugador()**.

Las clases internas tienen acceso a otras variables miembro de los objetos de la clase dentro de la que está definida aunque sean privados (se trata en cierto modo de un miembro más de la clase), mientras que las anidadas (estáticas) solo a las variables estáticas de la clase.

EJEMPLO: Para implementar el centro de un círculo, asociamos cada Circulo con su punto central definiendo la clase Punto como interna.

```
// Circulo.java
public class Circulo {

    class Punto {
        private int xPos, yPos;
        public Punto(int x, int y) { xPos = x;  yPos = y; }
        public String toString() { return "(" + xPos + "," + yPos + ")"; }
    }

    private Punto centro;
    private int radio;
    public Circulo(int x, int y, int r) {
         centro = this.new Punto(x, y);
         radio = r;
    }

    public String toString() {
        return "centro= " + centro + " y radio= " + radio;
    }

    public static void main(String []s) {
        System.out.println( new Circulo(10,10,20) );
    }
    // otros métodos...
}
```

Presta atención a como se instancia el objeto centro, que está declarada como private. Una limitación de las clases internas es que no pueden tener miembros estáticos:

```
class Externa {
    class Innterna { static int i = 10; }
}
Outer.java:3: inner classes cannot have static declarations
 static int i = 10;
```

CLASES INNER LOCALES

Las que **se definen dentro de un bloque de código**. No son miembros de una clase externa, sino locales al código donde se definen. Por tanto no son accesibles desde el exterior de ese código.

Ejemplo: Definir una clase inner local.

```
class AlgunaClase {
    void algunMetodo() {
        class Local { }  // inner local al definirse dentro del método
    }
}
```

Igual que las variables locales no pueden declararse como estáticas, las clases internas locales tampoco pueden ser estáticas.

Una interface no puede definir código, por tanto no puedes tener clases internas locales dentro de ellas. Y lo contrario también es cierto, no puedes definir interfaces locales.

EJEMPLO: tenemos esta clase abstracta:

```
abstract class Figura {

    public static class Color {
        int r, g, b;  // rojo, verde, azul

        public Color() { this(0, 0, 0); }
```

```
            public Color(int rp, int gp, int bp) { r = rp; g = gp;  b = bp; }

            public String toString(){ return "RGB=" + r + "," + g + "," + b; }
            // otros miembros...
        }
        // otros miembros...
    }
```

Imagina que en una clase Estado que utiliza la clase Figura, necesitas que el método toString() de Figura muestre una representación descriptiva de esta forma: "Color actual RGB=0,0,0". Debes definir un método llamado getEstado() en la clase Estado y creamos una clase derivada de Figura.Color:

```
class Estado {
   static Figura.Color getEstado(final Figura.Color c) {
       // La clase interna local definida dentro del método
       class ColorDescriptivo extends Figura.Color {
           public String toString() { return "Color actual " + c; }
       }
       return new ColorDescriptivo();
   }

   public static void main(String[] args) {
       Figura.Color cD = Estado.getEstado(new Figura.Color(0, 0, 0));
       System.out.println(cD);
    }
}
```

El método getEstado() acepta de parámetro una Figura y devuelve un objeto Figura.Color. Dentro del método se define la clase ColorDescriptivo que se deriva de la clase Figura.Color y define un único método que sobreecribe al método toString() de Figura. Después de definir la clase, crea un objeto de la clase y la devuelve.

Aunque no declares el parámetro como final, si intentas modificarlo en el código, se produce un error de compilación

cuando la clase interna la utilice.

CLASES INNER ANÓNIMAS

Algunas veces resulta útil crear clases que no tengan ni nombre, en una sola línea de código:

```
new superclase_o_interfaz(parámetros){ métodos_y_variables };
```

Este constructor define una nueva clase sin darle nombre y crea un objeto al mismo tiempo. La intención es crear un objeto personalizado de la superclase en el punto donde se ejecute. **Si se usa una interfaz en vez de una clase base, los parámetros deben estar vacíos, si se usa una clase base, los parámetros sirven para pasarlos al constructor.**

> **IMPORTANTE:** Para diferenciar cuando creas una clase anónima y la instancias de cuando simplemente creas una instancia de una clase, observa que en el primer caso además de llamar al constructor proporcionas un cuerpo donde la modificas: **new Clase(...){...};** mientras que al crear solo un objeto, no indicas el cuerpo (las llaves): **new Clase(...);**

Las clases anidadas anónimas se usan con mucha frecuencia en las aplicaciones GUI con los manejadores de eventos.

EJEMPLO: Considera la interfaz **Drawable**. Imagina que queremos un objeto Drawable que dibuje un cuadrado relleno de rojo de 100 píxels. En vez de definir una nueva clase separada en su fichero y luego usar esa clase para construir el objeto, podemos hacer lo mismo con una clase anónima:

```
Drawable cuadradoRojo = new Drawable() {
   void draw(Graphics g) { // Sobreescribo el método draw
      g.setColor(Color.RED);
      g.fillRect(10,10,100,100);
   }
}; // Este punto y coma es el final de la declaración
```

Las clases anónimas se usan frecuentemente para pasar parámetros actuales. Por ejemplo, el método siguiente dibuja un Drawable en dos contextos gráficos diferentes:

```
void dibuja2Veces( Graphics g1, Graphics g2, Drawable figura ) {
   figura.draw(g1);
   figura.draw(g2);
}
```

Cuando se haga una llamada, el tercer parámetro se puede crear en ese mismo momento usando una clase interna anónima. La posibilidad tradicional es crearla independiente con el esfuerzo y engorro que eso supone.

EJEMPLO: crear objetos para pasarlos como parámetro en el momento de realizar la llamada al método.

```
dibuja2Veces( primerG,
              segundoG,
              new Drawable() {
                    void draw(Graphics g) {
                        g.drawOval(10,10,100,100);
                    }
              }
            );
```

Cuando se compilan los fuentes, cada clase anónima genera un fichero class separado. Si el nombre de la clase principal es **UnNombre** por ejemplo, los ficheros .class de las clases anónimas serán **UnNombre$1.class**, **UnNombre$2.class**, **UnNombre$3.class**, etc.

5.8.2. EXPRESIONES LAMBDA.

La sintaxis de las clases anónimas es incómoda. Muchas veces implementan una clase que sigue una interfaz con solo un método. Java 8 introduce una nueva sintaxis que puede utilizarse en lugar de las clases anónimas en esa circunstancia: **una expresión lambda**. Esta es la expresión lambda equivalente a la llamada anterior:

```
dibuja2Veces( primerG, segundoG, g -> g.drawOval(10,10,100,100) )
```

La expresión lambda es **g -> g.drawOval(10,10,100,100)**. Su significado es: "el método que tiene un parámetro g y ejecuta el código **g.drawOval(10,10,100,100)**." El sistema deduce que g es de tipo Graphics porque espera un parámetro con la interfaz Drawable como parámetro actual, y el único método en la interfaz Drawable es un parámetro de tipo **Graphics**.

Las expresiones Lambda solamente pueden utilizarse en situaciones donde este tipo de inferencias se pueden realizar. La sintaxis general de una expresión lambda es:

parámetros_formales -> cuerpo_método

donde **cuerpo_método** puede ser una expresión sencilla, una llamada a subrutina o un bloque de sentencias: {sentencias }. Hay que tener en cuenta que:

- Cuando el cuerpo es una expresión sencilla o una llamada a función, el valor de la expresión se usa como valor de retorno del método que se ha definido.
- La lista de parámetros no debe especificar los tipos de datos, aunque puede hacerlo.
- Los paréntesis alrededor de la lista de parámetros son opcionales si solo hay un parámetro y no se indica tipo. De lo

contrario, o si no hay parámetros, los paréntesis son obligatorios.
- Para un método sin parámetros, la lista de parámetros es solo un par de paréntesis vacíos.

Puedes imaginar que una expresión lambda es una función anónima:
- Es anónima porque no tiene un nombre explícito como lo tiene un método o una clase o una variable.
- Es una función porque tiene una lista de parámetros, un cuerpo, un valor de retorno y una posible lista de excepciones que puede lanzar.
- Sin embargo, al contrario que una función normal, no tiene que pertenecer a una clase.
- Una expresión lambda puede pasarse como argumento a un método y puede ser devuelta como resultado.

Repasar la Sintaxis de las Expresiones Lambda:
Las expresiones lambda tienen 3 partes:
- Una lista de parámetros entre paréntesis (opcionales si solo tiene uno) y obligatorios si no hay parámetros. Ejemplo: **(File f)**
- Una flecha de dos caracteres: **->**
- Un cuerpo que es una sola sentencia o un bloque. Ej: **f.getName().endsWith(".xml")**. Por tanto podemos verlas:

  (parámetros) -> expresión;
  (parámetros) -> { varias_sentencias; }

Se puede omitir el tipo de los parámetros, puede inferirlo el compilador. Pero si indicas uno, debes indicarlos todos.

Las expresiones lambda son código, pero puede almacenarse en variables, por ejemplo en una variable de una **interfaz funcional** (una

interfaz de la que solo haya que implementar un método) y la expresión lambda equivale a ese método. Por ejemplo las interfaces siguientes:

```
@FunctionalInterface
public interface Runnable { void run(); }

@FunctionalInterface
public interface FileFilter { boolean accept(File pathname); }


Runnable r = () -> System.out.println("Hola");
FileFilter esXml = (File f) -> f.getName().endsWith(".xml");
```

Ejemplos de expresiones lambda:

```
() -> System.out.println("Hola mundo")
g -> { g.setColor(Color.RED); g.drawRect(10,10,100,100); }
(a, b) -> a + b;
(int n) -> {
               while (n > 0) {
                  System.out.println(n); n = n/2;
               }
            }; // lambda
```

5.8. EJERCICIOS.

E1. TEST

T1. Para declarar un objeto de una clase determinada, como atributo de otra clase, es necesario especificar que existe una relación de composición entre ambas clases mediante el modificador object. ¿Verdadero o Falso? **( )Verdadero** **( )Falso**

T2. Si se declaran dos variables objeto a y b de la clase X, ambas son instanciadas mediante un constructor, y posteriormente se realiza la asignación a=b, el contenido de b será una copia del contenido de a, perdiéndose los valores iniciales de b. ¿Verdadero o Falso?
( )Verdadero **( )Falso**

T3. Cuando escribas una clase en Java, puedes hacer que herede de una determinada clase padre (mediante el uso de **extends**) o bien no indicar ninguna herencia. En tal caso tu clase no heredará de ninguna otra clase Java. ¿Verdadero o Falso? **( )Verdadero ( )Falso**

T4. En Java los métodos heredados de una superclase deben volver a ser definidos en las subclases. ¿Verdadero o Falso?
( )Verdadero ( )Falso

T5. Dado que el método **finalize()** de la clase **Object** es **protected**, el método **finalize()** de cualquier clase que tú escribas podrá ser **public**, **private** o **protected**. ¿Verdadero o Falso? **( )Verdadero ( )Falso**

T6. Puede invocarse al constructor de una superclase mediante el uso de la referencia **this**. ¿Verdadero o Falso? **( )Verdadero ( )Falso**

T7. ¿Cuál de las siguientes características dirías que no es una de las que se suelen considerar como uno de los tres grandes pilares de la Programación Orientada a Objetos?

- Recursividad
- Herencia
- Polimorfismo
- Encapsulación

T8. El polimorfismo ofrece la posibilidad de que toda referencia a un objeto de una clase A pueda tomar la forma de una referencia a un objeto de cualquier otra clase B. ¿Verdadero o Falso? **( )Verdadero ( )Falso**

T9. El polimorfismo puede hacerse con referencias de superclases abstractas, superclases no abstractas o con interfaces. ¿Verdadero o Falso? **( )Verdadero ( )Falso**

T10. Una clase abstracta no podrá ser nunca instanciada. ¿Verdadero o Falso? **( )Verdadero** **( )Falso**

T11. Puede llamarse al constructor de una clase abstracta mediante el operador **new**. ¿Verdadero o Falso? **( )Verdadero** **( )Falso**

T12. Los métodos de una clase abstracta tienen que ser también abstractos. ¿Verdadero o Falso? **( )Verdadero** **( )Falso**

T13. Los modificadores **final** y **abstract** son excluyentes en la declaración de un método. ¿Verdadero o Falso? **( )Verdadero** **( )Falso**

T14. Una interfaz en Java no puede contener la implementación de un método mientras que una clase abstracta sí. ¿Verdadero o Falso?
( )Verdadero **( )Falso**

T15. En Java una clase no puede heredar de más de una clase abstracta ni implementar más de una interfaz. ¿Verdadero o Falso?
( )Verdadero **( )Falso**

T16. ¿Qué palabra reservada se utiliza en Java para indicar que una clase va a definir los métodos indicados por una interfaz?

- Implements
- uses
- extends

T17. Dada una clase Java que implementa dos interfaces diferentes que contienen un método con el mismo nombre, indicar cuál de las siguientes afirmaciones es correcta.

- Si los dos métodos tienen un valor de retorno de un tipo diferente, se producirá un error de compilación.
- Si los dos métodos tienen un valor de retorno de un tipo

diferente, se implementarán dos métodos.

- Si los dos métodos son exactamente iguales en identificador, parámetros y tipo devuelto, se producirá un error de compilación.
- Si los dos métodos tienen diferentes parámetros se producirá un error de compilación.

T18. En Java no está permitida la herencia múltiple ni para clases ni para interfaces. ¿Verdadero o Falso? **( )Verdadero ( )Falso**

RESPUESTAS:

E1. TEST:

T1: Falso. El hecho de declarar un atributo como de una clase determinada está ya indicando de manera implícita que existe una relación de composición (un objeto que contiene otros objetos en su interior) sin necesidad de tener que indicar nada más. Además, el modificador **object** no existe. Lo que sí existe es la clase **Object**, de la cual derivan todas las clases en Java.

T2: La variable b se habrá convertido en una referencia al mismo objeto al que apunta la variable a, pues no se ha hecho una copia de a. Ahora se tendrán dos variables referencia (a y b) a un mismo objeto (el que fue inicialmente instanciado y asignado a la variable b). Por otro lado, el objeto al que apuntaba la variable b se habrá perdido y el recolector de basura se encargará de destruirlo (salvo que existieran otras referencias a él).

T3: Falso. Aunque no indiques explícitamente ningún tipo de herencia, el compilador asume entonces de manera implícita que tu clase hereda de la clase Object, que define e implementa el comportamiento común a todas las clases.

T4: Falso. Los métodos heredados de la superclase están disponibles en la subclase sin tener que hacer nada especial, aunque es cierto que pueden ser redefinidos o ampliados (pero no es obligatorio)

T5: Falso. Sabemos que en Java toda clase acaba siempre derivando de la clase **Object** y también sabemos que cuando se redefine un método puede ampliarse su accesibilidad, pero nunca restringirse. De este modo, si el método original (el de la clase en la que se definió) es

protected, podrá ampliarse su accesibilidad a **public** o dejarse como **protected**, pero nunca restringirse más.

T6: Falso. La palabra reservada **this** es una referencia a la propia clase. Para hacer referencia a la superclase se utiliza la palabra reservada **super**.

T7: La recursividad existe en lenguajes no orientados a objetos (al margen de la POO)

T8: Falso. Para que eso pueda suceder la clase B debe ser subclase de A. En cualquier otro caso no funcionaría.

T9: Verdadero. Pueden declararse variables de un tipo superclase (abstracta o no) y más tarde instanciarse objetos subclase, o bien declararse variables de un tipo interfaz y posteriormente instanciarse objetos de clases que implementen esa interfaz.

T10: Por definición una clase abstracta es una clase que no se puede instanciar. Nunca podrán existir objetos de una clase abstracta. La idea es proporcionar un marco genérico para otras subclases más especializadas que deriven de ella. Pero por símisma no contiene suficiente información como para que existan instancias de ella.

T11: Falso. Una clase abstracta no puede ser instanciada y por tanto no puede llamarse a su constructor mediante el uso de new para crear un nuevo objeto basado en ella (es en cierto modo una clase "sin terminar"). Se produciría un error de compilación.

T12: Falso. Una clase abstracta puede contener métodos abstractos, pero también métodos que estén completamente definidos (no abstractos). Lo que sí es cierto es que si una clase contiene algún método abstracto, entonces tendrá que ser obligatoriamente una clase

abstracta.

T13. Veradero. No se puede ser a la vez **final** y **abstract**, pues si un método es **abstract** necesita ser redefinido, y un método **final** no permite serlo. Sería una contradicción.

T14. Verdadero. Una interfaz no puede definir métodos (no implementa su contenido), tan solo los declara. Sin embargo en una clase abstracta se pueden dejar algunos métodos sin implementar (métodos abstractos) e implementar otros.

T15. Falso. Aunque es cierto que solamente se puede heredar de una clase (sea abstracta o no), una clase Java sí puede implementar varias interfaces (tantas como se quiera).

T16. Implements.

T17. Se produce un error de compilación pues en la sobrecarga se utilizan los parámetros para distinguir unos métodos de otros, pero no el valor de retorno.

T18. Falso, no está permitida la herencia múltiple entre clases pero sí entre interfaces.

EA2. Define jerarquías de clases, o interfaces para los siguientes elementos:

- Las aeronaves pueden aterrizar y despegar. Las hay militares y civiles.
- Los aeropuertos tienen un nombre.
- Hay aeorpuertos terrestres que están en un país y una ciudad.
- También hay barcos que permiten despegar y aterrizar aeronaves.

- Un vuelo lo realiza un avión cuando despega y aterriza en el mismo o diferente lugar y está volando durante cierto tiempo.

EA3. Usa una clase abstracta llamada Figura2D que tenga los datos int x,y como su posición y los métodos abstractos perimetro() y area(). Crea las clases hija Rectángulo, Triángulo y Círculo.

EA4. Estamos desarrollando un juego y necesitamos una clase llamada Escenario que sea capaz de dibujar en un área de la ventana de una aplicación. Para ello, la hacemos hija de una clase llamada Component que le permite heredar el método paint(Graphics g) y dentro de él le permite dibujar. Como la queremos aprovechar para varios juegos, también debería implementar el método insertaPersonaje(Personaje p) y quizás algunos otros. No tenemos claro si crear una clase abstracta que los agrupe y hacer que Escenario sea una subclase suya o agruparlos en una interface ¿Cuál crees que sería la mejor opción? Haz un esqueleto de como quedarían definidos los elementos que aparecen en la descripción.

EA5. Tenemos las siguiente clases:

```
class Evaluacion {
        private int eval1, eval2, eval3;
        // Otros elementos
}

class Alumno {
        private String nombre;
        private Evaluación e;
        private double notaGlobal;

        public Alumno(String nombre, Evaluacion e) {
                this.nombre = nombre;
```

```
                this.e = e;
        }

        public Evaluacion getEvaluacion() { return e; }
        public void setEvaluacion(Evaluacion e) { this.e = e; }
        // Otros elementos...
}
```

a) Examina el código y piensa si hay alguna relación de composición o de agregación.
b) Puedes dibujar el diagrama de clases de UML de estas clases.
c) ¿Hay algún problema al usar la composición de esta manera?
d) Si lo hay indica qué elementos hay que modificar.
e) Haz las modificaciones necesarias.

EA6. Intentemos usar la palabra super:
La clase Mascota tiene las siguientes variables de instancia: int numPatas, String nombre, String voz y double peso. Crea la clase con un constructor donde se indiquen todos los atributos y getters y setters y sobreescribe tostring() para que devuelva "nombre numPatas voz peso". Además tiene el método come(double cantidad) que hace incrementar el peso de la mascota en cantidad y el método boolean juega(double minutos) que hace bajar el peso en minutos/2 siempre dejando el peso por encima de 0, o devuelve false si jugar tanto tiempo le hiciese perser el peso. El peso no tiene setter pero sí getter.

Crea la clase hija Perro que añade la variable de instancia color y usa la palabra super en el constructor y modifica el método juega para que si no quiere/puede jugar, antes coma minutos-1 cantidad y luego juegue. Usa super también en la implementación.

EA7. Usemos this y super en una clase que representa un Intervalo (un trozo) cerrado de la recta de números enteros

[menor, mayor]:

```
// Representa el intervalo [menor, mayor]
public class Intervalo {
    private int menor, mayor;

    public Intervalo(int menor, int mayor) {
          if( mayor < menor ) {
            int tmp = mayor;
            mayor = tmp;
            menor= tmp;
        }
    }
}
```

a) Añade la precondición en el constructor de que mayor debe ser distinto de menor.
b) Añade getters y setters manteniendo la precondición del apartado a
c) Añade el método boolean estaDentro(int x) que devuelva true cuando x está dentro del intervalo o false en caso contrario.
d) Queremos añadir a esta clase un constructor por defecto (sin parámetros) que genere a partir de un LocalDateTime menor y mayor:

d1 <- la hora del día + segundos * 10
d2 <- el mes del año * minutos

Pero queremos aprovechar el constructor anterior. ¿Cómo podrías hacerlo?
e) Creamos la interface Localizable que obliga a implementar el método "boolean estaDentro(int x)".
f) Haz que la clase intervalo la implemente.
g) Define la subclase IntervaloAbierto que añade dos variables de instancia:

boolean abiertoAbajo, abiertoArriba.

h) Añade constructor por defecto para que sea abierto por arriba y por abajo y aproveche el constructor por defecto de la clase padre.

i) Crea el constructor con todos los atributos y aprovecha el contructor de la clase padre.
j) Modifica el método "estaDentroDe(int x)" para que en caso de que el método del padre indique que pertenece, comprobemos si es abierto por abajo que x es mayor estricto que menor. Y si es abierto por arriba, que x sea menor estricto que mayor.

EA8. Escoge entre clase, clase abstracta e interfaz y crea el esqueleto de las siguientes cosas que hay en una granja teniendo en cuenta que:

- subclase ES UN/UNA superclase -> Herencia
- supeclase no sabe como sus hijos harán algo, pero quiere que lo hagan -> declara pero no implementa algún comportamiento -> clase abstracta.
- clase DEBE SER CAPAZ DE hacer cosas -> Interface

Lo que nos vamos a encontrar en la granja de mi tía es:

- Seres vivos que no hacen la fotosíntesis ni se reproducen por esporas y tienen dos variables de instancia de esta forma: "LocalDateTime nacimiento" y "String formaDeNacer" con el método "void dimeNacesDe()" y se delega la implementación.
- Es capaz de volar (tiene el método void vuela() que imprime como vuela).
- Vaca.
- Oveja.
- Gallina ponedora.
- Perro.
- Paloma
- Gato.
- Tigre.
- MulaMecanica (una especie de pequeño tractor).

- Canario
- Es un animal mascota (acompaña) dentro de la casa: habla()
- Es un animal que vive con el hombre porque le es útil (trabaja, proporciona alimento, etc.)
- Desempeña un trabajo (void trabaja()).

EA09. Crea expresiones lambda para las siguientes interfaces funcionales:

a)

```
interface I1 {
  public void carga(int Kg);
  default public boolean cabeMas() {
       return true;
  }
}
```

b)

```
interface I2 {
  public int maximo(int n1, int n2);
        public String toString();
  }
```

EA10. La interface java.io.FileFilter de java permite crear un filtro para descartar nombres de ficheros. Esta interface funcional declara el método:

boolean accept(File f);

Adapta el siguiente ejemplo que llama al método listFiles(ficherosJava) donde ficherosJava es un objeto de una clase que implemente FileFilter y que devuelva true cuando el nombre del fichero (getName()) acabe en '.java' y false en otro caso. Así consigues un array de nombres de ficheros fuente de la carpeta actual. La clase MiFiltro la defines como una clase pública externa en su propio fichero .java:

```
import java.io.*;

public class EA10 {
   MiFiltro ficherosJava = new MiFiltro(".java");
   // Hace un listado de los ficheros que acaban en .java de la
```

```
        // carpeta actual
        static public void main(String[] args) throws IOException {
            Files[] fuentes = new File(".").listFiles( ficherosJava );
            for(int i= 0; i < fuentes.length; i++)
                 System.out.println( fuentes[i] );
         }
    }
```

EA11. Copia el fichero EA10.java en el fichero EA11.java y vuelve a implementa de nuevo el mismo escenario, pero ahora crea una clase no pública (en el mismo fichero que EA11.java).

EA12. Copia el fichero EA11.java en el fichero EA12.java pero ahora la clase MiFiltro que sea una clase clase interna a la clase EA12.

EA13. Copia el fichero EA12.java en el fichero EA13.java y ahora usa una clase clase anónima como filtro.

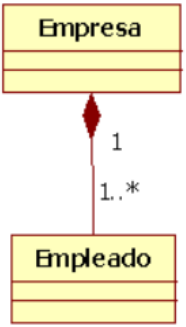
EA14. Copia el fichero EA13.java en el fichero EA14.java y ahora usa una función lambda para usarla como filtro.

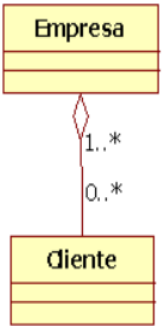
EJERCICIOS DEL TEMA

EJERCICIO 1: Crea el constructor de copia de la clase Rectangulo.

```
public Rectangulo(Punto vertice1, Punto vertice2) {
   this.vertice1 = new Punto( vertice1 );
   this.vertice2 = new Punto( vertice2 );
}
```

EJERCICIO 2: Observa estos dos gráficos UML e indica dónde se usa la composición y donde la agregación. Intenta implementarlas indicando constructores que reflejen las relaciones.






```
// Empresa está compuesto de Empleados
// Si una empresa desaparece sus empleados también
// Es decir, no puede compartirlos con el exterior
// Por tanto atención en constructores, getters y setters!!
public class Empresa {
   // sus cosas...

  public Empresa(...) { /* copio Objetos */ }
  public void contrataEmpleado(Empleado e) { /* copio e */ }
  public Empleado getEmpleado(int e){ return empleados[i].copia; }
}
```

```
// EN el caso de la derecha Empresa agrega varios Cliente
// Si una empresa desaparece sus clientes no
// Es decir, puede compartirlos con el exterior, salvo que...
// Pierdas restricciones de acceso!!
// En ese caso cuidado en constructores, getters y setters
public class Empresa {
   // sus cosas...
   public Empresa(...) { /* copio Objetos */ }
   public void nuevoCliente(Ciente c) { /* asigno o copio c */ }
   public Empleado getCliente(int i){
      if (no_pierdo_restricciones ) return clientes[i];
      else return clientes[i].copia();
  }
}
```

EJERCICIOS PROPUESTOS.

E1. TEST

E1. TEST:

T1: Falso. El hecho de declarar un atributo como de una clase determinada está ya indicando de manera implícita que existe una relación de composición (un objeto que contiene otros objetos en su interior) sin necesidad de tener que indicar nada más. Además, el modificador **object** no existe. Lo que sí existe es la clase **Object**, de la cual derivan todas las clases en Java.

T2: **Falso.** La variable b se habrá convertido en una referencia al mismo objeto al que apunta la variable a, pues no se ha hecho una copia de a. Ahora se tendrán dos variables referencia (a y b) a un mismo objeto (el que fue inicialmente instanciado y asignado a la variable b). Por otro lado, el objeto al que apuntaba la variable b se habrá perdido

y el recolector de basura se encargará de destruirlo (salvo que existieran otras referencias a él).

T3: Falso. Aunque no indiques explícitamente ningún tipo de herencia, el compilador asume de manera implícita que tu clase hereda de la clase Object, que define e implementa el comportamiento común a todas las clases.

T4: **Falso**. Los métodos heredados de la superclase están disponibles en la subclase sin tener que hacer nada especial, aunque es cierto que pueden ser redefinidos o ampliados (pero no es obligatorio)

T5: **Falso**. Sabemos que en Java toda clase acaba siempre derivando de la clase **Object** y también sabemos que cuando se redefine un método puede ampliarse su accesibilidad, pero nunca restringirse. De este modo, si el método original (el de la clase en la que se definió) es **protected**, podrá ampliarse su accesibilidad a **public** o dejarse como **protected**, pero nunca restringirse más.

T6: **Falso**. La palabra reservada **this** es una referencia a la propia clase. Para hacer referencia a la superclase se utiliza la palabra reservada **super**.

T7: La recursividad existe en lenguajes no orientados a objetos (al margen de la POO).

T8: Falso. Para que eso pueda suceder la clase B debe ser subclase de A. En cualquier otro caso no funcionaría.

T9: Verdadero. Pueden declararse variables de un tipo superclase (abstracta o no) y más tarde instanciarse objetos subclase, o bien declararse variables de un tipo interfaz y posteriormente instanciarse objetos de clases que implementen esa interfaz.

T10: Por definición una clase abstracta es una clase que no se puede instanciar. Nunca podrán existir objetos de una clase abstracta. La idea es proporcionar un marco genérico para otras subclases más especializadas que deriven de ella. Pero por sí misma no contiene suficiente información como para que existan instancias de ella.

T11: Falso. Una clase abstracta no puede ser instanciada y por tanto no puede llamarse a su constructor mediante el uso de new para crear un nuevo objeto basado en ella (es en cierto modo una clase "sin terminar"). Se produciría un error de compilación.

T12: Falso. Una clase abstracta puede contener métodos abstractos, pero también métodos que estén completamente definidos (no abstractos). Lo que sí es cierto es que si una clase contiene algún método abstracto, entonces tendrá que ser obligatoriamente una clase abstracta.

T13. Verdadero. No se puede ser a la vez **final** y **abstract**, pues si un método es **abstract** necesita ser redefinido, y un método **final** no permite serlo. Sería una contradicción.

T14. Verdadero. Una interfaz no puede definir métodos (no implementa su contenido), tan solo los declara. Sin embargo en una clase abstracta se pueden dejar algunos métodos sin implementar (métodos abstractos) e implementar otros.

T15. Falso. Aunque es cierto que solamente se puede heredar de una clase (sea abstracta o no), una clase Java sí puede implementar varias interfaces (tantas como se quiera).

T16. Implements.

T17. Se produce un error de compilación pues en la sobrecarga se utilizan los parámetros para distinguir unos métodos de otros, pero no el valor de retorno.

T18. Falso, no está permitida la herencia múltiple entre clases pero sí entre interfaces.

EA2. Define jerarquías de clases, o interfaces para los siguientes elementos:

- Las aeronaves pueden aterrizar y despegar. Las hay militares y civiles.
- Los aeropuertos tienen un nombre.
- Hay aeorpuertos terrestres que están en un país y una ciudad.
- También hay barcos que permiten despegar y aterrizar aeronaves.
- Un vuelo lo realiza un avión cuando despega y aterriza en el mismo o diferente lugar y está volando durante cierto tiempo.

```
interface Aeronave {
    public void despega();
    public void aterriza();
}

public class AvionCivil implements Aeronave {
    public void despega();
    public void aterriza();
}

public class AvionMilitar implements Aeronave {
    public void despega();
    public void aterriza();
}

public class Aeropuerto {   private String nombre;    }
```

```
public class AeropuertoTerrestre extends Aeropuerto {
        private String pais, ciudad;
}

public class Barco extends Aeropuerto {  }

public class Vuelo {
    Aeropuerto origen;
    double tiempoVuelo;
    Aeropuerto destino;
}
```

EA3. Usa una clase abstracta llamada Figura2D que tenga los datos int x,y como su posición y los métodos abstractos perimetro() y area(). Crea las clases hija Rectángulo, Triángulo y Círculo.

```
public abstract class Figura2D {
    private int x, y;
    abstract public double perimetro();
    abstract public double area();
    // ...
}

public class Rectangulo extends Figura2D {
    private int lado1, lado2;

    public double perimetro() { return 2 * (lado1 + lado2);  }
    public double area() {  return lado1 * lado2; }
    // ...
}

public class Triangulo extends Figura2D {
    private int lado1, lado2, lado3;

    public double perimetro() { return lado1 + lado2 + lado3;  };
    public double area() {
          double s = perimetro()/2;
          return Math.sqrt( (s-lado1) * (s-lado2) * (s-lado3) );
    }
    // ...
```

```
}

public class Circulo extends Figura2D {
    private int radio;

    public double perimetro() { return 2 * Math.PI * radio;  };
    public double area() {
            return Math.PI * radio * radio;
    }
    // ...
}
```

EA4. Estamos desarrollando un juego y necesitamos una clase llamada <u>Escenario</u> que sea capaz de dibujar en un área de la ventana de una aplicación. Para ello, la hacemos hija de una clase llamada <u>Component</u> que le permite heredar el método paint(Graphics g) y dentro de él le permite dibujar. Como la queremos aprovechar para varios juegos, también debería implementar el método insertaPersonaje(Personaje p) y quizás algunos otros. No tenemos claro si crear una clase abstracta que los agrupe y hacer que Escenario sea una subclase suya o agruparlos en una interface ¿Cuál crees que sería la mejor opción? Haz un esqueleto de como quedarían definidos los elementos que aparecen en la descripción.

Como en Java una clase solamente puede heredar de una superclase, y Escenario necesita heredar de Component para dibujar, no podemos usar la opción de la clase abstracta, por tanto debemos elegir agrupar los métodos que debe implementar un Escenario en una interface. Quedaría así:

```
public class Personaje { }

public interface Espectaculo {
   public void insertaPersonaje(Personaje p);
}
```

```
public class Escenario extends Component implements Espectaculo {
    // ..

    // Para pintar sobreescribe el método de su padre
    @Override
    void paint(Graphics g) { ... }

    // Implementa el/los métodos de las interfaces que implementa
    public void insertaPersonaje(Personaje p) { ... }
}
```

EA5. Tenemos las siguientes clases:

```
class Evaluacion {
        private int eval1, eval2, eval3;
        // Otros elementos
}

class Alumno {
        private String nombre;
        private Evaluacion e;
        private double notaGlobal;

        public Alumno(String nombre, Evaluacion e) {
                this.nombre = nombre;
                this.e = e;
        }

        public Evaluacion getEvaluacion() { return e; }
        public void setEvaluacion(Evaluacion e) { this.e = e; }
        // Otros elementos
}
```

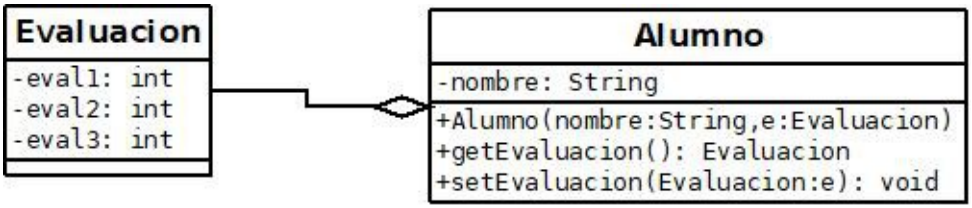
a) Examina el código y piensa si hay alguna relación de composición o de agregación.

Un objeto alumno está formado por un objeto Evaluación (es una de sus partes). Tal y como aparece es una agregación porque si un alumno dejase de existir, su evaluación podría seguir existiendo. Ejemplo:

```
// Desde otra clase
```

```
Evaluacion e = new Evaluacion(6, 7, 8);
Alumno a = new Alumno("pepe", e);
a = null;   // a deja de existir pero el objeto al que
            // referencia a.e que es el mismo al que referencia
            // e sigue existiendo!!
    :
```

b) Puedes dibujar el diagrama de clases de UML de estas clases.



c) ¿Hay algún problema al usar la composición de esta manera?

Sí, puede haber dos fallos:

- Si es una composición en vez de una agregación lo que queríamos implementar, no está conseguida.
- Además, no conseguimos asegurar el modificador private de la variable de instancia e de la clase Alumno.

d) Si lo hay indica qué elementos hay que modificar.

Podemos perder el private al compartir el objeto con el exterior por la forma de implementar la clase Alumno en:

- El constructor.
- El getter
- El setter

e) Haz las modificaciones necesarias.

Hay que evitar asignar a un objeto interno las referencias que nos llegan del exterior (en el constructor y en setter) y no devolver al

exterior referencias directas a los objetos internos que se puedan cambiar (que no sean inmutables).
La mejora manera de solucionarlo es fabricando un constructor de copia (recibe un objeto y fabrica una copia del mismo) o un método de copia o clonado (aunque no sea un constructor). Voy a implementar 3 soluciones:

```
// *** PRIMERA: constructor de copia en Evaluacion
class Evaluacion {
        private int eval1, eval2, eval3;

        public Evaluacion( Evaluacion original ) {
                eval1 = original.eval1;
                eval2 = original.eval2;
                eval3 = original.eval3;
        }
}

// En la clase Alumno modifico los métodos problemáticos
public Alumno(String nombre, Evaluacion e) {
        this.nombre = nombre;
        this.e = new Evaluacion(e);  // guardo una copia
}

public Evaluacion getEvaluacion() {
        return new Evaluacion(e);  // Devuelvo una copia
}

public void setEvaluacion(Evaluacion e) {
        this.e = new Evaluacion(e);
}

// *** SEGUNDA: un método de instancia llamado copia
class Evaluacion {
        private int eval1, eval2, eval3;

        public Evaluacion copia() {
            return new Evaluacion(eval1, eval2, eval3);
        }
}
```

```
// En la clase Alumno modifico los métodos problemáticos
public Alumno(String nombre, Evaluacion e) {
        this.nombre = nombre;
        this.e = e.copia();  // guardo una copia
}

public Evaluacion getEvaluacion() {
        return e.copia();  // Devuelvo una copia
}

public void setEvaluacion(Evaluacion e) {
        this.e = e.copia();
}


// *** TERCERA: método a nivel de clase copia
class Evaluacion {
        private int eval1, eval2, eval3;

        static public Evaluacion copia( Evaluacion e ) {
                return new Evaluacion(e.eval1, e.eval2, e.eval3);
        }
}

// En la clase Alumno modifico los métodos problemáticos
public Alumno(String nombre, Evaluacion e) {
        this.nombre = nombre;
        this.e = Evaluacion.copia(e);  // guardo una copia
}

public Evaluacion getEvaluacion() {
        return new Evaluacion.copia(e);  // Devuelvo una copia
}

public void setEvaluacion(Evaluacion e) {
        this.e = Evaluacion.copia(e);
}
```

Nota: investiga el método clone() de Object.

EA6. Intentemos usar la palabra super:

La clase Mascota tiene las siguientes variables de instancia: int

numPatas, String nombre, String voz y double peso. Crea la clase con un constructor donde se indiquen todos los atributos y getters y setters y sobreescribe tostring() para que devuelva "nombre numPatas voz peso". Además tiene el método come(double cantidad) que hace incrementar el peso de la mascota en cantidad y el método boolean juega(double minutos) que hace bajar el peso en minutos/2 siempre dejando el peso por encima de 0, o devuelve false si jugar tanto tiempo le hiciese perder todo su peso. El peso no tiene setter pero sí getter.

```
public class Mascota {
    private int numPatas;
    private String nombre;
    private String voz;
    private double peso;

    public Mascota(int pats, String nom, String voz, double pes) {
       numPatas = pats; nombre = nom; this.voz = voz; peso = pes;
    }

    public int getNumPatas() { return numPatas; }
    public String getNombre() { return nombre; }
    public String getVoz() { return voz; }
    public double getPeso() { return peso; }

    public void setNumPatas(int n) { numPatas = n; }
    public void setNombre(String n) { nombre = n; }
    public void setVoz(String v) { voz = v; }

    public void come(double cantidad) {   peso+= cantidad; }

    public boolean juega(double minutos) {
         double perdida = minutos/2;
         if (peso – perdida > 0) {
             peso -= perdida;
             return true;
         }
         return false;
    }
}
```

Crea la clase hija Perro que añade la variable de instancia color y usa la palabra super en el constructor y modifica el método juega para que si no quiere/puede jugar, antes coma minutos-1 cantidad y luego juegue minutos/2. Usa super también en la implementación.

```
public class Perro extends Mascota {
    private String color;

    public Perro(int pats, String nom, String voz, double pes,
                 String color) {
        super(pats, nom, voz, pes);  // Construyo Mascota
        this.color = color;
    }

    public boolean juega(double minutos) {
         boolean haJugado= super.juega(minutos);
         if( !haJugado ) {
              come(minutos - 1);
              return super.juega(minutos/2);
         }
         return true;
    }
}
```

EA7. Usemos this y super en una clase que representa un Intervalo (un trozo) cerrado de la recta de números enteros [menor, mayor]:

```
// Representa el intervalo [menor, mayor]
public class Intervalo {
    private int menor, mayor;

    public Intervalo(int menor, int mayor) {
         if( mayor < menor ) {
           int tmp = mayor;
           mayor = tmp;
           menor = tmp;
        }
    }
}
```

a) Añade la precondición en el constructor de que mayor debe ser distinto de menor.

```
public Intervalo(int mayor, int menor) {
    if( mayor == menor )
       throw new IllegalArgumentException("Extremos iguales");
    if( mayor < menor ) {
       int tmp = mayor;
       mayor = menor;
       menor = tmp;
    }
}
```

b) Añade getters y setters manteniendo la precondición del apartado a

```
public int getMenor() { return menor; }

public int getMayor() { return mayor; }

public void setMenor(int men) {
    if (men < mayor) menor = men;
}

public void setMayor(int may) {
    if (may > menor) mayor = may;
}
```

c) Añade el método boolean estaDentro(int x) que devuelva true cuando x está dentro del intervalo o false en caso contrario.

```
public boolean estaDentro(int x) {
    return menor <= x && x <= mayor;
}
```

d) Queremos añadir a esta clase un constructor por defecto (sin parámetros) que genere a partir de un LocalDateTime menor y mayor:

d1 <- la hora del día + segundos * 10

d2 <- el mes del año * minutos

Pero queremos aprovechar el constructor anterior. ¿Cómo podrías hacerlo?

Si queremos aprovechar el constructor que tiene los dos parámetros enteros habría que usar de primera sentencia en el nuevo constructor su llamada: "this(int, int);" Pero claro, esos dos parámetros hay que generarlos, por tanto necesitaríamos sentencias anteriores ¿Cómo hacerlo? Bueno, podemos llevarnos el código anterior necesario para rellenar las dos variables parámetro a métodos de uso interno (privado) y llamarlos desde dentro del this o guardarlos en otras variables de usa privado. Como no pueden ser iguales o lanzaremos una excepción, opto por un solo método privado que rellene el primer parámetro y una variable de instancia privada para el segundo:

```
private static int paramD2;

private static int generaParametros() {
    int d1, d2;
    do {
        LocalDateTime ldt = LocalDateTime.now();
        d1 = ldt.getHour() + ldt.getSecond() * 10;
        d2 = ldt.getMonthValue() * ldt.getMinute();
    }
    while ( d1 == d2 );
    paramD2 = d2;
    return d1;
}

public Intervalo() {
   this(Intervalo.generaParametros(), Intervalo.paramD2);
}
```

e) Creamos la interface Localizable que obliga a implementar el método "boolean estaDentro(int x)".

```
public interface Localizable {
    public boolean estaDentro(int x);
}
```

f) Haz que la clase Intervalo la implemente.

```
public class Intervalo implements Localizable {
    private int menor, mayor;
    // lo anterior...
    // El método de la interface ya estaba
    public boolean estaDentro(int x) {
        return menor <= x && x <= mayor;
    }
}
```

g) Define la subclase IntervaloAbierto que añade dos variables de instancia:

boolean abiertoAbajo, abiertoArriba.

```
public class IntervaloAbierto extends Intervalo{
    private int menor, mayor;
    private boolean abiertoAbajo, abiertoArriba;
}
```

h) Añade constructor por defecto para que sea abierto por arriba y por abajo y aproveche el constructor por defecto de la clase padre.

```
    public IntervaloAbierto() {
        super();
        abiertoAbajo = abiertoArriba = true;
    }
```

i) Crea el constructor con todos los atributos y aprovecha el contructor de la clase padre.

```
public IntervaloAbierto(int me, int ma, boolean abMe, boolean
abMa) {
    super(me, ma);
    abiertoAbajo = abMe; abiertoArriba = abMa;
}
```

j) Modifica el método estaDentro(int x) para que en caso de que el método del padre indique que pertenece, comprobemos si es abierto por abajo que x es mayor estricto que menor. Y si es abierto por arriba, que x sea menor estricto que mayor.

```
public boolean estaDentro(int x) {
    if( super.estaDentro(x) ) {
      if( abiertoAbajo && x == menor ) return false;
      if( abiertoArriba && x == mayor ) return false;
      return true;
    }
    return false;
}
```

EA8. Escoge entre clase, clase abstracta e interfaz y crea el esqueleto de las siguientes cosas que hay en una granja teniendo en cuenta que:

- subclase ES UN/UNA superclase -> Herencia
- supeclase no sabe como sus hijos harán algo, pero quiere que lo hagan -> declara pero no implementa algún comportamiento -> clase abstracta.
- clase DEBE SER CAPAZ DE hacer cosas -> Interface

Lo que nos vamos a encontrar en la granja de mi tía es:

- Seres vivos que no hacen la fotosíntesis ni se reproducen por esporas y tienen dos variables de instancia de esta forma: "LocalDateTime nacimiento" y "String formaDeNacer" con el método "void dimeNacesDe()" del que se delega la implementación.
- Es capaz de volar (tiene el método void vuela() que imprime como vuela).
- Vaca.
- Oveja.
- Gallina ponedora.
- Perro.
- Paloma
- Gato.

- Tigre.
- MulaMecanica (una especie de pequeño tractor).
- Canario
- Es un animal mascota (acompaña) dentro de la casa: habla()
- Es un animal que vive con el hombre porque le es útil (trabaja, proporciona alimento, etc.)
- Desempeña un trabajo (void trabaja()).

```
public abstract class Animal {
    private LocalDateTime nace;
    private String formaDeNacer;

    abstract public void dimeNacesDe();
}

public interface Volador {  public void vuela(); }

public interface Mascota {  public void habla(); }

public interface Trabaja {  public void trabaja(); }

public abstract class  AnimalDomestico extends Animal {  }

public class Vaca extends AnimalDomestico{
    public void dimeNacesDe() { System.out.println("parto"); }
}

public class Oveja extends AnimalDomestico{
    public void dimeNacesDe() { System.out.println("parto"); }
}

public class GallinaPonedora extends AnimalDomestico implements
Trabaja {
   public void dimeNacesDe() { System.out.println("huevo"); }
   public void trabaja() { System.out.println("pongo un huevo"); }
}

public class Perro extends AnimalDomestico implements Trabaja,
Mascota {
   public void dimeNacesDe() { System.out.println("parto"); }
```

```
        public void trabaja() { System.out.println("pongo un huevo"); }
        public void habla() { System.out.println("guau"); }
    }

    public class Paloma extends AnimalDomestico implements Volador {
        public void dimeNacesDe() { System.out.println("huevo"); }
        public void vuela() { System.out.println("mucho"); }
    }

    public class Gato extends AnimalDomestico implements Trabaja,
    Mascota {
        public void dimeNacesDe() { System.out.println("parto"); }
        public void trabaja() { System.out.println("cazo roedor"); }
        public void habla() { System.out.println("miau"); }
    }

    public class Tigre extends Animal {
        public void dimeNacesDe() { System.out.println("parto"); }
    }

    public class Mulamecanica implements Trabaja {
        public void trabaja() { System.out.println("labro"); }
    }

    public class Canario extends AnimalDomestico implements Volador,
    Mascota {
        public void dimeNacesDe() { System.out.println("huevo"); }
        public void habla() { System.out.println("pio, pio"); }
        public void vuela() { System.out.println("si abres jaula"); }
    }
```

EA9. Crea expresiones lambda para las siguientes interfaces funcionales:

a)

```
interface I1 {
  public void carga(int Kg);
  default public boolean cabeMas() {
        return true;
  }
}
```

b)

```
interface I2 {
 public int maximo(int n1, int n2);
        public String toString();
}
```

a) Es funcional porque cabeMas() tiene implementación por defecto. Nefesitamos una función lambda que no devuelve nada y acepta un entero de parámetro:

b) Es funcional porque toString lo hereda de Object, hay que implementar el primero que devuelve un int y y acepta de parámetros dos enteros.

```
public class EA09 {

   static public void main(String[] args) {
      I1 una = (i) -> System.out.println( "Cargo " + i + " Kg");
      una.carga(5);
      I2 dos = (i1, i2) -> {  int max;
                              if( i1 > i2 ) max = i1;
                              else max= i2;
                              return max;
                           };
      System.out.println( dos.maximo(8, 15) );
   }
}

interface I1 {
    public void carga(int Kg);
    default public boolean cabeMas() {
       return true;
    }
}

interface I2 {
    public int maximo(int n1, int n2);
       public String toString();
}
```

EA10. La interface java.io.FileFilter de java permite crear un filtro para descartar nombres de ficheros. Esta interface funcional declara el método:

boolean accept(File f);

Adapta el siguiente ejemplo que llama al método listFiles(ficherosJava) donde ficherosJava es un objeto de una clase que implemente FileFilter y que devuelva true cuando el nombre del fichero (getName()) acabe en '.java' y false en otro caso. Así consigues un array de nombres de ficheros fuente de la carpeta actual. La clase MiFiltro la defines como una clase pública externa en su propio fichero .java:

```
import java.io.*;

public class EA10 {
   MiFiltro ficherosJava = new MiFiltro(".java");
   // Hace un listado de los ficheros que acaban en .java de la
   // carpeta actual
   static public void main(String[] args) throws IOException {
       Files[] fuentes = new File(".").listFiles( ficherosJava );
       for(int i= 0; i < fuentes.length; i++)
            System.out.println( fuentes[i] );
    }
}

// Fichero MiFiltro.java
import java.io.*;

public class MiFiltro implements FileFilter {
    private String sufijo;

    public MiFiltro(String s) { sufijo = s; }

    // Implementa la interfaz FileFilter
    public boolean accept(File f) {
        return  f.getName().endsWith(sufijo);
    }
}
```

EA11. Copia el fichero EA10.java en el fichero EA11.java y vuelve a implementa de nuevo el mismo escenario, pero ahora crea una clase no pública (en el mismo fichero que EA11.java).

```
// Fichero EA11.java
```

```
import java.io.*;

public class E10 {

   // Hace un listado de los ficheros que acaban en .java de la
   // carpeta actual
   static public void main(String[] args) throws IOException {
       MiFiltro ficherosJava = new MiFiltro(".java");
       File[] fuentes = new File(".").listFiles( ficherosJava );
       for(int i= 0; i < fuentes.length; i++)
            System.out.println( fuentes[i] );
    }
}

class MiFiltro implements FileFilter {
    private String sufijo;

    public MiFiltro(String s) { sufijo = s; }

    public boolean accept(File f) {
        return  f.getName().endsWith(sufijo);
    }
}
```

EA12. Copia el fichero EA11.java en el fichero EA12.java pero ahora la clase MiFiltro que sea una clase clase interna a la clase EA12.

```
// Fichero EA12.java
import java.io.*;

public class EA12 {

    // Debe ser static porque la referenciamos desde main
    static class MiFiltro implements FileFilter {
        private String sufijo;
        public MiFiltro(String s) { sufijo = s; }
        public boolean accept(File f) {
            return f.getName().endsWith(sufijo);
        }
    }

   // Hace un listado de los ficheros que acaban en .java de la
   // carpeta actual
```

```
    static public void main(String[] args) throws IOException {
        MiFiltro ficherosJava = new MiFiltro(".java");
        File[] fuentes = new File(".").listFiles( ficherosJava );
        for(int i= 0; i < fuentes.length; i++)
             System.out.println( fuentes[i] );
     }
}
```

EA13. Copia el fichero EA12.java en el fichero EA13.java y ahora usa una clase clase anónima como filtro.

```
// Fichero EA13.java
// Observa como ahora el código es más corto
import java.io.*;

public class EA13 {
   // Hace un listado de los ficheros que acaban en .java de la
   // carpeta actual
   static public void main(String[] args) throws IOException {
       File[] fuentes = new File(".").listFiles(
                           new FileFilter() {
                             public boolean accept(File f) {
                              return f.getName().endsWith(".java");
                             }
                           }  // fin de la clase/objeto
                       );
       for(int i= 0; i < fuentes.length; i++)
           System.out.println( fuentes[i] );
    }
}
```

EA14. Copia el fichero EA13.java en el fichero EA14.java y ahora usa una función lambda para usarla como filtro.

```
import java.io.*;

public class EA14 {
   // Hace un listado de los ficheros que acaban en .java de la
   // carpeta actual
   static public void main(String[] args) throws IOException {
      // También podemos pasarla directamente en el parámetro
      FileFilter ff = (f) -> f.getName().endsWith(".java");
```

```
        File[] fuentes = new File(".").listFiles( ff );
        for(int i= 0; i < fuentes.length; i++)
             System.out.println( fuentes[i] );
    }
}
// La variación pasando la expresión lambda
import java.io.*;

public class EA14A {
   static public void main(String[] args) throws IOException {
       File[] fuentes = new File(".")
              .listFiles( (f) -> f.getName().endsWith(".java") );
       for(int i= 0; i < fuentes.length; i++)
            System.out.println( fuentes[i] );
    }
}
```